Audit

You are a senior system auditor reviewing an AI-native platform.

Context:
<describe feature / workflow / API>

Your task:
1. Identify loopholes, edge cases, and failure points
2. Classify them (Security / Workflow / UX / Data / AI)
3. Explain how they can be exploited or triggered
4. Estimate impact (Low / Medium / High)
5. Suggest concrete fixes

Be brutally critical. Assume users will misuse the system.

I audited the current repo implementation, since the feature/workflow spec was left as a placeholder. The biggest issues are not subtle; several are outright account-compromise or cross-tenant data-exposure paths.

1. Email-based admin backdoor
Class: `Security`
Impact: `High`
Trigger/exploit: any user can register `support@joallm.ai`, because registration trusts arbitrary emails and there is no verification step in [`auth.ts` at registration](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L37) and the platform auto-upgrades that email to admin/enterprise in [`access-control.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/access-control.ts#L31) and again during auth hydration in [`auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/middleware/auth.ts#L226). That same email also gets extra workflow debug leakage in [`workflows.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/workflows.ts#L59).
Fix: remove all email-string privilege overrides; gate support access with a server-managed role only; require verified email ownership before account activation.

2. Unauthenticated RAG can search the global corpus
Class: `Security / Data / AI`
Impact: `High`
Trigger/exploit: both `/api/rag/search` and `/api/rag/chat` allow `optionalAuth` in [`rag.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/rag.ts#L66) and [`rag.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/rag.ts#L966). When no `fileIds` are supplied, the search layer only filters on `files.status='processed'` and does not scope by owner/workspace in [`enhanced-rag-service.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/enhanced-rag-service.ts#L156). An unauthenticated caller can probe the entire processed knowledge base and get raw chunk text back.
Fix: require auth for all RAG endpoints, or add mandatory tenant/user/workspace scoping in the search queries themselves. Never rely on “fileIds were omitted” as a safe mode.

3. Public workflows can exfiltrate private documents
Class: `Security / Data / Workflow / AI`
Impact: `High`
Trigger/exploit: public workflows are executable by other users in [`workflows.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/workflows.ts#L446). But workflow node execution trusts embedded `fileIds` and raw `fileId` values without re-checking ownership in the `rag` and `media_ingest` handlers in [`workflow-executor.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-executor.ts#L490) and [`workflow-executor.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-executor.ts#L531). A user can publish a workflow referencing their private file IDs, and any authenticated user who executes it can force retrieval against those assets.
Fix: public workflows must not retain executable references to private resources; every node that touches files/documents must re-authorize against the caller's access scope at execution time.

4. Refresh tokens are effectively non-revocable session skeleton keys
Class: `Security`
Impact: `High`
Trigger/exploit: refresh tokens are self-contained JWTs in [`auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/middleware/auth.ts#L132), refresh just verifies signature in [`auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L269), logout blacklists only the presented access token in [`auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L550), and password reset does not revoke prior refresh tokens in [`auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L794). Stolen refresh tokens remain usable until expiry even after logout or password reset.
Fix: store refresh tokens server-side with token IDs, rotate on every use, revoke on logout/password change/password reset, and bind them to device/session records.

5. Frontend "secure storage" is just reversible localStorage obfuscation
Class: `Security / UX`
Impact: `High`
Trigger/exploit: auth and refresh tokens are stored in localStorage in [`AuthContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/AuthContext.tsx#L165) and the "encryption" key is hardcoded client-side in [`storage.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/utils/storage.ts#L5). Any XSS, malicious extension, or same-origin script can decrypt everything instantly.
Fix: move auth to `HttpOnly`, `Secure`, `SameSite` cookies; remove client-side reversible token storage; treat any browser-stored bearer token as already compromised.

6. Google integration state can be forged and tokens are stored plaintext
Class: `Security / Data`
Impact: `High`

Trigger/exploit: the Google Workspace OAuth state token falls back to `'dev-secret'` in [`integrations.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/integrations.ts#L24), which makes state forgery realistic in any misconfigured deploy. The same route then writes `accessToken` and `refreshToken` directly to the DB in plaintext in [`integrations.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/integrations.ts#L121).
Fix: fail closed if the JWT secret is missing; use a dedicated OAuth state secret; encrypt third-party OAuth tokens at rest with envelope encryption; rotate and redact them from logs and exports.

7. Declared security headers are not actually applied
Class: `Security`
Impact: `Medium`
Trigger/exploit: the app imports `securityHeaders` in [`index.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/index.ts#L15) and comments that they are registered, but no hook/plugin actually applies them; the only global hook shown is raw-body capture in [`index.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/index.ts#L106). The header definitions sit unused in [`security.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/middleware/security.ts#L27).
Fix: install a real security-header middleware/plugin, verify headers in integration tests, and do not rely on comments as evidence of enforcement.

8. File upload path is memory- and queue-amplifiable
Class: `Workflow / UX / Security`
Impact: `Medium-High`
Trigger/exploit: non-media uploads are buffered fully in memory in [`files.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts#L755), then shoved into the document queue payload in [`files.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts#L815). Original file storage can also persist large blobs in queue processing in [`queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts#L341). A few concurrent 50 MB uploads across accounts can pressure Node memory, Redis, and worker throughput hard.
Fix: stream uploads directly to object storage, queue only references, enforce per-user and global byte quotas, and add concurrency caps/backpressure.

9. User export likely over-exposes sensitive internals
Class: `Data / UX`
Impact: `Medium`
Trigger/exploit: `/user-settings/export` selects the full `users` row and returns it verbatim in [`user-settings.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/user-settings.ts#L245). That likely includes password hashes, encrypted API keys, internal metadata, and fields the user does not need in a browser-delivered export.

Fix: build an explicit export DTO; exclude password hashes, encrypted secrets, reset/session/security fields, and internal operational metadata by default.

10. Secret-bearing files and logs are committed in the repo tree
Class: `Security / Workflow`
Impact: `Medium`
Trigger/exploit: the repo currently contains [`services/backend/.env`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/.env), [`services/backend/.env.local`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/.env.local), [`services/backend/logs/combined.log`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/logs/combined.log), and [`services/backend/logs/error.log`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/logs/error.log). Even if values are currently benign, this is how real secrets, tokens, and PII eventually leak.
Fix: purge them from version control, rotate anything that may have been exposed, add strict `.gitignore` rules, and scan CI for secrets before merge.

Most urgent fixes, in order: kill the email-based admin override, lock down RAG to authenticated tenant-scoped access, block public workflow access to private resources, and replace the refresh-token model. Until those are fixed, I would not consider this platform safe for hostile or enterprise use.

Analysis

Codebase Understanding Prompt

Codebase Understanding Prompt

Act as a principal software architect analyzing this codebase.

Your goal is to build a deep understanding of the system.

Analyze the codebase and explain:

1. What this system does at a business level
2. What the main modules/services are
3. How the request flow works end-to-end
4. What the key architectural patterns are
5. What the main dependencies and infrastructure assumptions are
6. Which parts appear mature vs fragile
7. What appears to be missing or inconsistent

Return the response in these sections:
- Business Purpose
- System Architecture
- Core Modules
- Request/Execution Flows
- Design Patterns
- Risks / Weak Signals
- Questions the codebase raises

Do not just summarize files. Infer how the system actually works.

Business Purpose

This system is trying to be an AI work platform rather than a single chatbot product. At the business level, it combines five monetizable capabilities into one account and data model:

- Multi-model chat over commercial and local LLMs
- Knowledge ingestion and retrieval-augmented generation over user files
- Visual workflow automation using AI/data-processing nodes
- Notebook-style exploratory work with AI-aware cells
- Enterprise-style controls around roles, tiers, usage, security, feedback, and integrations

The product shape is closer to an “AI operating surface” for individuals, teams, and enterprise users than a narrow assistant. The recurring theme is: capture user context once, then let that context flow into chat, RAG, workflows, notebooks, bookmarks, and training signals.

A second business goal is visible too: the platform is designed to learn from usage. Message feedback, copy/regenerate signals, RAG grounding lineage, training consent, and

usage/cost tracking suggest the team wants both product analytics and future model-training/evaluation datasets.

System Architecture

The live codebase is effectively a modular monolith with multiple frontends, not a finished microservice platform.

The clearest deployed architecture is:

- `services/backend`: one Fastify API that owns most business logic
- `services/frontend`: the authenticated product UI
- `services/landing-page`: the marketing site
- `shared/`: cross-service types/utilities, though in practice it looks lighter than the backend's own domain code
- PostgreSQL: primary system of record, including vector search via pgvector
- Redis: cache, queue transport, token exchange helper, and optional async-processing backbone
- BullMQ workers: background document/media/workflow work
- Object/local storage: file originals and derived media artifacts
- Prometheus/Grafana: observability layer

There is also an older or aspirational microservices story in [`docker-compose.microservices.yml`](/Users/aeishwary/JoaLLM-platform/joallm-platform/docker-compose.microservices.yml), but the implemented backend is a consolidated API registered through [`services/backend/src/domains/index.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/domains/index.ts). That's an important architectural truth: the code talks like "multi-service enterprise platform," but the implementation center of gravity is one backend process plus workers.

The data model reinforces that. The backend owns users, organizations/workspaces, chat sessions, messages, files, document chunks, workflows, notebooks, bookmarks, training/feedback, subscriptions, integrations, and audit/security state in one schema file starting at [`services/backend/src/database/schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts). That means product cohesion is high, but service boundaries are not yet real boundaries.

Core Modules

The main modules are product domains more than technical layers.

- Auth and identity
  - Registration, login, refresh, logout, Google OAuth, 2FA, sessions, password reset.
  - It also drives role/tier projection and lightweight enterprise scaffolding.
  - This is the foundation for all downstream access checks.

- Access, subscription, and preferences
  - Roles control permissions.
  - Subscription tiers control limits/features.
  - Workspace mode is partly UX state and partly a signal toward future tenant-aware behavior.
  - `/api/me/access` is the bootstrap endpoint that lets the frontend hydrate “who am I, what can I do, what are my limits?”

- Chat
  - Standard multi-turn LLM chat with session history, parameter tuning, streaming, exports, bookmarks, and implicit feedback.
  - Chat is not isolated from the rest of the platform; it can pull in selected RAG documents and can write chat output back into the knowledge base.

- Files and knowledge
  - Upload, store, parse, chunk, embed, reprocess, delete, status-poll, transcript, and media analysis.
  - This is the substrate for RAG, media intelligence, and some workflow nodes.
  - The file system supports both classic documents and media assets.

- RAG
  - Search, chunk inspection, query enhancement, analytics, RAG sessions, and RAG chat.
  - It is implemented as a service layer over uploaded files and document chunks, not a separate service boundary.
  - It is a hybrid of keyword and vector retrieval with confidence heuristics.

- Workflow automation
  - User-authored node/edge graphs persisted as JSONB.
  - Executions are stored as first-class records.
  - Some nodes run inline, some suspend and resume through queues.
  - This is the most “agentic orchestration” part of the system.

- Notebooks
  - Structured workspace for mixed cell types including AI, chart, knowledge, agent, debug.
  - This looks like the product’s exploratory/research environment.

- Feedback / telemetry / training lineage
  - Tracks explicit and implicit signals, API usage, RAG grounding, audit logs, metrics.
  - This is both an ops layer and a data flywheel layer.

- Third-party integrations
  - Currently at least Google Workspace OAuth plumbing.
  - Indicates the platform wants to ingest/work with external systems, not only local uploads.

On the frontend, the product is organized as a workspace shell around these modules. [`services/frontend/src/App.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/App.tsx) shows the UI as one authenticated app with route-based feature areas rather than separate products.

Request/Execution Flows

The system has a few important end-to-end flows.

1. App bootstrap
- User opens frontend.
- Auth context pulls token/user from browser storage.
- Frontend calls auth/profile and `/api/me/access`.
- Backend authenticates JWT, hydrates current DB role, then returns role/tier/permissions/limits/workspace mode.
- Frontend uses that to gate UI and shape feature affordances.

This is important because the frontend depends heavily on a server-supplied access snapshot, not just the token.

2. Chat flow
- Frontend sends message history, chosen model, parameters, optional selected document IDs.
- Backend authenticates and may resolve selected docs into accessible context.
- Chat orchestrator calls LLM provider abstraction.
- Response, token usage, session/message history, and usage/cost telemetry are persisted.
- Additional side effects include auto-title/session completion signals, bookmarks, and message feedback.

This flow is not just “send prompt to model.” It’s really: session state + optional retrieval + model call + usage accounting + training signal capture.

3. Document-to-RAG flow
- User uploads file.
- Backend validates type/size and creates a file record immediately.
- If queue is available, processing is pushed to background; otherwise parts can run synchronously/fallback.
- Document processor extracts text, adaptive chunker creates chunks, embeddings are generated, chunks land in pgvector-backed tables.
- File metadata/status are updated as the pipeline advances.
- Later, RAG search or RAG chat queries those chunks.

This is a pipeline architecture disguised as CRUD. The file row is the workflow anchor, and statuses/metadata are being used as pipeline state.

4. RAG search / RAG chat flow
- Frontend calls RAG search or chat endpoints.
- Backend may validate requested file IDs against the caller.
- Query is sanitized, optionally enhanced, then sent through hybrid retrieval.
- Retrieved chunks become either direct search results or context for LLM synthesis.
- Search history, session history, lineage, and usage are recorded.

The interesting architectural point: RAG is treated as both a search product and a context-enrichment service for other product surfaces.

5. Workflow execution flow
- User builds graph in frontend and saves nodes/edges.
- Backend stores workflow definition as data.
- On execute, backend creates a `workflowExecutions` row.
- Execution is queued if workers are available, else run in-process.
- Executor topologically walks the DAG, parallelizes ready nodes, and can suspend on async media jobs.
- Resume happens from a persisted checkpoint when async work completes.
- Outputs/logs/status are written back to execution records.

This is one of the more sophisticated flows in the codebase. It is moving toward durable orchestration rather than a simple request/response API.

6. Media intelligence flow
- Media file upload stores original asset.
- Background jobs extract metadata/audio/frames/transcript/insights.
- Derived artifacts and transcript segments are persisted.
- Those outputs can feed RAG or workflow nodes.

This suggests the platform is evolving from “document RAG” into “multimodal knowledge operations.”

Design Patterns

Several architectural patterns show up repeatedly.

- Modular monolith by domain
  - Backend is split into auth/chat/files/rag/workflow/etc. domains with route/service/repository helpers.
  - This is a good fit for current complexity and team size.

- API-first frontend/backend separation
  - Frontend consumes explicit endpoint catalogs like [`services/frontend/src/config/api.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/config/api.ts).
  - Product UI is clearly a client of backend contracts.

- Orchestrator pattern
  - Chat and RAG chat have application-layer orchestrators.
  - They compose retrieval, model invocation, lineage, and persistence.

- Pipeline/state-machine pattern for file processing
  - File rows, chunk rows, queue jobs, and metadata flags together form a document/media processing pipeline.
  - The database is partly being used as the workflow state store.

- Durable execution / checkpoint-resume
  - Workflow execution persists checkpoints and resumes after async jobs.
  - That is more advanced than typical CRUD-heavy SaaS code.

- Provider abstraction
  - LLM providers, embedding generation, and storage providers are abstracted behind service interfaces.
  - This supports multi-vendor AI and multiple storage backends.

- Server-side entitlement model
  - Permissions and limits are computed server-side and exposed as a single access snapshot.
  - That's a solid pattern for complex product gating.

- Observability woven into business paths
  - Metrics, audit logs, usage costs, lineage tables, and feedback capture are not afterthoughts.
  - The system is designed to observe itself and its users heavily.

Risks / Weak Signals

Some parts look mature; others look like fast-moving platform code that hasn't fully converged.

More mature
- The product vision is coherent. Chat, files, RAG, workflows, notebooks, and feedback are not random features; they share users, files, sessions, and AI lineage.
- The backend has a real domain map and persistent execution records.
- The data model is rich and clearly designed for future enterprise features.
- Background processing and fallback behavior are thought through in several places.
- The workflow engine and RAG orchestration show genuine architectural ambition.

More fragile
- The codebase tells two architecture stories at once: a modular monolith and a microservice platform. The monolith is real; the microservices compose file looks partly aspirational. That kind of split often causes duplicated assumptions and stale docs.
- Tenant/workspace support appears only partially integrated. The schema includes organizations/workspaces/memberships, but much of the product still looks primarily user-scoped.
- Some features feel "stitched in" rather than normalized. Media AI, training datasets, subscriptions, integrations, and workspace scaffolding all exist, but not always with the same depth of consistency as core chat/files.
- Frontend and landing-page duplication suggests parallel evolution or migration drift.
- The shared package exists, but the backend still carries a lot of local type/service logic; shared boundaries may be weaker than intended.
- Security, access, and storage patterns are not uniformly mature across all surfaces, which usually means the platform grew feature-first.

Missing or inconsistent
- The business model is richer than the enforcement model. Tiers/roles/workspaces exist, but the access semantics do not yet feel uniformly tenant-native.
- The “AI-native platform” story includes training data collection and enterprise scaffolding, but there is not yet a fully obvious governance model for data lifecycle, retention, export scope, or isolation.
- The deployment artifacts suggest more decomposition than the code presently implements.
- The notebook subsystem is present, but compared with chat/RAG/workflows it feels less obviously integrated in the backend architecture map.
- Some docs and filenames imply legacy iterations or alternate implementations rather than one settled architecture.

Questions the codebase raises

- Is the intended long-term architecture a modular monolith, or are chat/auth/files/RAG meant to become real services later? Right now the codebase seems to be in between.
- What is the true tenancy model: single-user with optional team overlays, or workspace-first enterprise isolation? The schema points one way, many request paths still point another.
- Are workflows and notebooks first-class execution runtimes, or mostly product shells around existing chat/RAG services?
- How much of the “training data layer” is analytics, and how much is intended to feed fine-tuning/evaluation pipelines?
- Is media intelligence a core product line, or an expansion branch attached to the document-RAG architecture?
- Why are there both `frontend` and `landing-page` implementations with overlapping patterns, and are they intentionally separate products or just separate surfaces?
- Which API contracts are stable versus transitional? The presence of compatibility fields and legacy endpoints suggests the frontend and backend may still be converging.
- How much of the enterprise story is operationally real today? Organizations, workspaces, integrations, audit logs, subscriptions, and support overrides exist, but not all with the same architectural maturity.

If you want, I can turn this into a second pass that maps the system as a diagram: frontend, backend domains, queues, storage, DB tables, and the major flows between them.

Monolith vs Services Reality Check

Monolith vs Services Reality Check

Analyze this codebase and determine its true architectural shape.

Do not rely on folder names or developer intent. Infer from the actual code.

Evaluate:
1. Is this a monolith, modular monolith, service-oriented system, or pseudo-microservice architecture?
2. Which modules are actually decoupled and which are tightly coupled?
3. What shared dependencies create coupling?
4. Where are the hidden boundaries?
5. Which parts could be separated into services with minimal pain?
6. Which parts would be dangerous to split right now?

Return:
- Current Architecture Type
- Evidence from Code
- Coupling Hotspots
- Logical Service Boundaries
- Refactor Readiness Assessment

Current Architecture Type

This is a `modular monolith with a worker sidecar`, plus some `pseudo-microservice residue`.

It is not a true service-oriented system in the running code. The real runtime shape is:

- one backend API process in [`services/backend/src/index.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/index.ts)
- one optional queue worker process in [`services/backend/src/worker.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/worker.ts)
- one frontend app calling that backend directly
- one shared PostgreSQL schema and one shared Redis/queue layer used by nearly every backend capability

The “microservice” compose file is mostly an architectural sketch, not the implemented backbone. The actual code path is a single Fastify app registering all domains in-process via [`services/backend/src/domains/index.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/domains/index.ts).

So the truest label is: `modular monolith, partially preparing for future service extraction, with some pseudo-microservice documentation/deployment artifacts`.

Evidence from Code

1. **Single runtime entrypoint**
- The backend boots one Fastify server and registers all domains locally in [`services/backend/src/index.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/index.ts) and [`services/backend/src/domains/index.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/domains/index.ts).
- There is no inter-service RPC between auth/chat/files/rag/workflows inside the live backend.

2. **One shared database connection for all domains**
- Everything imports the same `db` from [`services/backend/src/database/connection.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/connection.ts).
- The schema is centralized in one file starting at [`services/backend/src/database/schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts).
- Chat, auth, files, workflows, notebooks, RAG, preferences, subscriptions all read/write the same database directly.

3. **Cross-domain in-process imports**
- RAG chat orchestrator imports retrieval, model service, repositories, and mode config directly in [`services/backend/src/application/rag/rag-chat-orchestrator.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/application/rag/rag-chat-orchestrator.ts).
- Chat routes directly import `ragService`, `llmService`, DB schema, and application orchestrator in [`services/backend/src/routes/chat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts).
- Workflow executor directly calls `ragService`, `llmService`, DB tables, and queue primitives in [`services/backend/src/services/workflow-executor.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-executor.ts).
- Queue workers import document processing, embeddings, RAG indexing, workflow runner, media processing, DB schema, and storage in one place in [`services/backend/src/services/queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts).

4. **Domain wrappers are thin routing shells, not isolation boundaries**
- The “domain” modules are mostly just route registration plus auth wrappers, e.g. [`services/backend/src/domains/chat/index.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/domains/chat/index.ts), [`services/backend/src/domains/files/index.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/domains/files/index.ts), [`services/backend/src/domains/rag/index.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/domains/rag/index.ts).
- The actual business logic still lives in shared routes/services/repositories over common infrastructure.

5. **Worker is separate process, not separate service boundary**
- There is a distinct queue worker entrypoint in [`services/backend/src/worker.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/worker.ts).
- But it runs the same codebase, same DB, same Redis, same services, same schema.
- That is runtime separation for throughput, not architectural decoupling.

6. **Pseudo-microservice artifacts are not authoritative**
-
[`docker-compose.microservices.yml`](/Users/aeishwary/JoaLLM-platform/joallm-platform/docker-compose.microservices.yml) describes `auth-service`, `chat-service`, `api-gateway`, etc.
- But the actual repo's main scripts run `services/backend`, `services/frontend`, and `services/landing-page` from one workspace root in [`package.json`](/Users/aeishwary/JoaLLM-platform/joallm-platform/package.json).
- The backend package is singular and owns the broad API surface in [`services/backend/package.json`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/package.json).

Coupling Hotspots

1. **Database schema coupling**
- This is the biggest coupling anchor.
- Most modules bypass any anti-corruption layer and query shared tables directly.
- Splitting services would require untangling shared ownership over `users`, `files`, `chatSessions`, `messages`, `documentChunks`, `workflows`, `workflowExecutions`, `userPreferences`, `userSecurity`, etc.

2. **Auth and entitlement coupling**
- Authentication, role/tier resolution, and permission enforcement are embedded everywhere through common middleware and DB lookups.
- Even when modules look separate, they depend on shared auth semantics and user rows.

3. **Files as a platform-wide substrate**
- Files are not just a file service concern.
- RAG, media intelligence, chat save-to-knowledge, workflows, notebooks, rendering, transcripts, and insights all depend on file state and file-derived artifacts.
- That makes `files` and `documentChunks` a central dependency hub.

4. **Shared AI provider layer**
- `llmService`, embedding generation, BYOK handling, and usage tracking are reused across chat, RAG, workflows, and notebooks.
- This creates beneficial reuse, but it means the AI execution layer is not domain-local.

5. **Queue orchestration as shared control plane**
- The queue layer processes document indexing, media pipelines, and workflow execution together in

[`services/backend/src/services/queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts).
- That means background execution concerns are coupled across product areas.

6. **Workflow engine as a cross-domain consumer**
- Workflows directly consume LLM, RAG, files, media pipelines, and lineage.
- That makes workflows a high-coupling aggregator and one of the hardest pieces to extract safely.

7. **Frontend contract drift**
- Frontend calls one backend directly via endpoint config in [`services/frontend/src/config/api.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/config/api.ts).
- There is no BFF/API gateway abstraction in the active code path, so any backend decomposition would ripple into the frontend unless a stable facade is introduced first.

Logical Service Boundaries

These are the boundaries that exist logically, even if they are not enforced yet.

1. **Identity / Access**
- Users, auth, security, tiering, preferences bootstrap, integrations.
- This could become a service because it already acts as the platform authority for identity and entitlements.
- But it still shares tables and assumptions with many product flows.

2. **Conversation / Chat**
- Chat sessions, messages, streaming, message feedback, title generation, session completion signals.
- This is a reasonably coherent domain, especially if document selection is treated as an external dependency.

3. **Knowledge / Retrieval**
- Files, document processing, chunking, embeddings, RAG search, RAG chat lineage.
- This is one of the clearest service candidates because it already behaves like a pipeline-backed domain.
- The caution is that workflows, notebooks, and chat all reach into it.

4. **Workflow Orchestration**
- Workflow definitions, executions, checkpoint/resume, orchestration runtime.
- Logically separable, but operationally it is a dependency aggregator, so separation is harder than it looks.

5. **Notebook Runtime**
- Notebook entities and cell execution.
- Conceptually separate, but currently implemented partly as a consumer of shared LLM/RAG primitives rather than a self-contained subsystem.

6. **Telemetry / Lineage / Audit**
- Metrics, audit logs, usage, inference lineage, training signals.
- This is a hidden but important boundary: it spans all product surfaces and could become platform infrastructure rather than business-domain logic.

Where the hidden boundaries are
- `application/*` orchestrators are closer to true boundaries than `domains/*`.
- `repositories/*` hint at a few emerging seams, especially around API keys, RAG access, and lineage.
- The worker/API split is a hidden operational boundary.
- The file-processing pipeline is a stronger boundary than the route layout suggests.
- The access bootstrap endpoint is effectively a policy/entitlement boundary.

Refactor Readiness Assessment

Best candidates to split with relatively low pain
1. **Telemetry / metrics / audit / lineage**
- High fan-in but low business entanglement.
- Can often be peeled off behind async events or append-only writes first.

2. **Model catalog / model metadata**
- `models` is likely one of the lowest-risk extractions because it is read-heavy and less entangled with transactional state.

3. **Document processing worker path**
- Not the whole knowledge domain, but the extraction/chunking/embedding pipeline could be isolated first because it already runs asynchronously.
- The API could keep file ownership/status while the worker side becomes a more service-like processor.

4. **Media processing**
- Media extraction, transcription, frame analysis, rendering already behave like specialized async workloads.
- This is a natural compute boundary if the file/storage contract is stabilized.

Could be split, but only after preparatory work
1. **Knowledge/RAG as a whole**
- Logical boundary is strong.
- Practical coupling is still high because shared tables and direct DB access are everywhere.
- First step would be replacing direct table reads with internal facades/events/contracts.

2. **Chat**
- Chat could be isolated if session/message persistence, auth context, usage tracking, and retrieval dependencies are mediated cleanly.
- Right now it still reaches across common infra too freely.

3. **Identity/access**

- Important boundary, but dangerous to extract prematurely because auth and entitlement checks are embedded throughout routes and middleware.

Dangerous to split right now
1. **Workflow orchestration**
- It reaches into LLM, RAG, files, media queues, lineage, and execution state all at once.
- Extracting it now would force distributed transactions or a lot of immature cross-service contracts.

2. **Anything requiring ownership of `users` plus feature data**
- Since many modules directly read `users`, `userPreferences`, `userSecurity`, and feature tables together, naive extraction would create chatty inter-service auth/profile calls everywhere.

3. **Frontend-facing API decomposition without a gateway/BFF**
- The frontend is coded against one backend surface.
- Splitting backend services now would push orchestration complexity to the client unless a stable gateway is introduced first.

Bottom line
- `Current state`: modular monolith with worker sidecar.
- `Not true microservices`: domain wrappers and folders do not correspond to deployable, independently evolving services.
- `Real seams`: async processing, telemetry/lineage, media pipeline, model catalog, and eventually knowledge/RAG.
- `Most dangerous split today`: workflow execution and core auth-entitlement logic, because they currently depend on too much shared state and too many in-process assumptions.

If you want, I can follow this with a concrete extraction roadmap: “what to split first, in what order, and what interfaces/contracts need to exist before each move.”

End to End flow extraction prompt

Act as a systems analyst.

Trace the most important end-to-end flows in this codebase.

For each major user journey or system workflow:
1. Entry point
2. Validation layer
3. Business logic layer
4. Persistence layer
5. Async/background processing
6. External integrations
7. Final response or system state change

Prioritize the most critical flows first.

Output format:
- Flow Name
- Trigger / Entry Point
- Sequence of Components Involved
- State Changes
- Failure Points
- Notes on Hidden Complexity

Focus on actual execution paths, not theoretical intent.

- **Flow Name**
  User Authentication: Register / Login / Refresh / 2FA / Google OAuth

- **Trigger / Entry Point**
  - `POST /api/auth/register`, `POST /api/auth/login`, `POST /api/auth/refresh` in [`auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L839)
  - `POST /api/security/2fa/verify` in [`security.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/security.ts#L322)
  - `GET /api/auth/google/callback` in [`auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L584)

- **Sequence of Components Involved**
  - Route handler parses request with Zod schemas in [`auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L17)
  - Validation/rate limiting via `EndpointValidations.*` and auth rate-limit config
  - Auth business logic checks `users` and `userSecurity`, hashes passwords with `bcrypt`, creates JWTs via middleware helpers

  - 2FA login path issues a `pending_2fa` token in [`auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L197), then `/2fa/verify` upgrades it to full access in [`security.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/security.ts#L376)
  - Google callback exchanges auth code through [`googleAuthService.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/googleAuthService.ts), upserts user data, generates JWTs, then stores them temporarily in Redis before redirecting with a one-time code in [`auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L662)

- **State Changes**
  - New `users` row on register or first-time Google login
  - `userSecurity.failedLoginAttempts` / `lockedUntil` updated on failed login
  - `userSecurity.twoFactor*` updated for 2FA setup/verification
  - Redis entry `oauth_code:*` created for OAuth token handoff
  - Tokens returned to client; later auth middleware re-hydrates user role/email from DB in [`middleware/auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/middleware/auth.ts#L226)

- **Failure Points**
  - DB unavailable when creating/fetching users
  - Redis unavailable breaks OAuth code exchange fallback path
  - 2FA secret missing or backup codes exhausted
  - Refresh token invalid/expired
  - Google token exchange or ID-token verification failure

- **Notes on Hidden Complexity**
  - Auth is not just identity; it also projects role/tier overrides and immediate entitlement effects
  - `pending_2fa` is enforced by auth middleware, so 2FA is partially modeled as a temporary auth state, not just a flag
  - OAuth callback is coupled to Redis and frontend redirect behavior, not purely backend-local

- **Flow Name**
  Standard Chat Request

- **Trigger / Entry Point**
  - `POST /api/chat/send` in [`chat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts#L846)

- **Sequence of Components Involved**

  - Request hits `authenticateToken` and `requireModelForTier()` prehandlers in [`chat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts#L847)
  - Body parsed through `ChatRequestSchema`
  - Route calls `runChat()` in [`application/chat/chat-orchestrator.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/application/chat/chat-orchestrator.ts#L148)
  - `runChat()` gets/creates a `chatSessions` row, persists the user message, fetches BYOK provider keys through `userApiKeyRepository`, then calls `llmService.generateResponse()`
  - Assistant response is persisted back to `messages`
  - Auto-title generation runs fire-and-forget through a second LLM call in [`chat-orchestrator.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/application/chat/chat-orchestrator.ts#L106)

- **State Changes**
  - New `chatSessions` row when session is absent
  - New user `messages` row, then new assistant `messages` row
  - `chatSessions.title` may be updated asynchronously
  - Token usage is stored on assistant message

- **Failure Points**
  - Session creation can succeed even if later LLM call fails, leaving partial conversation state
  - Provider call can fail due to model/provider/key issues
  - Title-generation can fail independently after successful response
  - BYOK decryption or retrieval can fail and silently degrade to platform key usage/fallback behavior

- **Notes on Hidden Complexity**
  - The clean API hides a two-phase persistence pattern: user message is written before the model call
  - Title generation is a second AI side effect, not part of the core response path
  - Chat is tightly coupled to sessions, provider abstraction, BYOK, and usage capture even in the “simple” path

- **Flow Name**
  Streaming Chat Request

- **Trigger / Entry Point**
  - `POST /api/chat/stream` in [`chat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts#L944)

- **Sequence of Components Involved**
  - Same auth/tier prehandlers as sync chat

  - Route sets raw SSE headers directly on `reply.raw` in [`chat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts#L998)
  - Session is created or loaded in-line rather than via `runChat()`
  - User message is persisted
  - Streaming provider path emits token chunks to the client
  - On completion, assistant message and usage are persisted

- **State Changes**
  - `chatSessions` may be created
  - User message persisted before stream starts
  - Assistant message persisted only after successful completion of the stream
  - Session state exists even if client disconnects mid-stream

- **Failure Points**
  - Client disconnect during stream
  - Provider stream errors after user message was already saved
  - CORS/header handling differs from normal backend response path
  - Partial output may have been sent to client without corresponding assistant persistence

- **Notes on Hidden Complexity**
  - This path duplicates significant session/persistence logic instead of reusing `runChat()`
  - Streaming changes transaction semantics: the client can observe output before the DB reflects final state
  - Error handling is harder because network, model, and persistence failures can happen in different phases

- **Flow Name**
  File Upload to Searchable Knowledge Asset

- **Trigger / Entry Point**
  - `POST /api/files/upload` in [`files.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts#L532)

- **Sequence of Components Involved**
  - Auth + permission prehandlers: `authenticateToken`, `requirePermission('knowledge.write')`
  - Multipart parsing via Fastify multipart
  - Route validates MIME/extension/size, reads full buffer, creates a `files` row in status `uploaded` in [`files.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts#L758)
  - Audit log written
  - If queue is available:
    - document files go to `document-processing`
    - media files are first stored, then `extract-metadata` is enqueued

  - Document-processing worker in [`queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts#L314) extracts text via `documentProcessor`, chunks it, inserts `documentChunks`, updates `files.metadata`, and then enqueues `document-indexing`
  - Document-indexing worker calls `ragService.indexDocument()` in [`queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts#L510), which generates embeddings and updates chunk rows

- **State Changes**
  - `files` row created immediately
  - `files.status` progresses through `uploaded` → `processing` → `processed` or `failed`
  - `documentChunks` rows inserted
  - Chunk embeddings added later by indexing worker
  - `files.metadata.processingStage/indexingStatus` are used as workflow state

- **Failure Points**
  - Large buffer read in API process before queue handoff
  - Queue enqueue failure marks file failed
  - Extraction failure after file row already exists
  - Storage upload can fail but document processing may continue
  - Indexing can fail after chunk insertion, leaving keyword-only searchability

- **Notes on Hidden Complexity**
  - This is a multi-stage pipeline disguised as one upload endpoint
  - `files.metadata` acts like a state machine
  - Searchability is incremental: processed text may exist before vectors do
  - Queue availability changes system behavior materially

- **Flow Name**
  RAG Search

- **Trigger / Entry Point**
  - `POST /api/rag/search` in [`rag.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/rag.ts#L66)

- **Sequence of Components Involved**
  - `optionalAuth` + RAG validation prehandler
  - Body parsed with `RAGSearchRequestSchema`
  - If `fileIds` are supplied and user is authenticated, ownership is checked against `files`
  - Query is sanitized
  - Cache lookup via `cacheService`
  - Core retrieval via `enhancedRAGService.search()` in [`enhanced-rag-service.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/enhanced-rag-service.ts)
  - Service may do query enhancement, vector search, keyword search, or hybrid merge
  - Search history, optional session query logging, and API usage accounting are persisted

  - Result is cached asynchronously

- **State Changes**
  - `searchHistory` row inserted
  - `ragSearchQueries` row may be inserted if `sessionId` is supplied
  - `apiUsage` row may be inserted for authenticated users
  - Cache entry created

- **Failure Points**
  - Cache miss then retrieval failure
  - Embedding generation can fail, causing vector fallback behavior
  - Session logging can fail without failing the user-visible request
  - Ownership checks only happen when explicit `fileIds` are provided

- **Notes on Hidden Complexity**
  - RAG search is both a user feature and a telemetry pipeline
  - The search result path includes caching, logging, costing, and session analytics
  - Hybrid retrieval can partially succeed and degrade silently, which makes behavior harder to reason about than a single search algorithm

- **Flow Name**
  RAG Chat

- **Trigger / Entry Point**
  - `POST /api/rag/chat` in [`rag.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/rag.ts#L967)

- **Sequence of Components Involved**
  - `optionalAuth` prehandler
  - Route forwards to `runRagChat()` in [`rag-chat-orchestrator.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/application/rag/rag-chat-orchestrator.ts#L102)
  - Orchestrator resolves mode config, validates document accessibility if document IDs were provided, fetches user API keys, and calls `enhancedRAGService.searchWithConfidence()`
  - For research mode, it may do a second retrieval pass using expansion terms from first-pass results
  - Retrieved results are formatted into context via `rag-modes`
  - `llmService.generateResponse()` synthesizes an answer, or the system falls back to a structured non-LLM response
  - `ragLineageRepository` records success/failure, prompt/response context, sources, and usage

- **State Changes**
  - Lineage/inference records persisted for success or failure
  - Source linkage for the response is captured in repository-level writes
  - No chat-session persistence here; this is a parallel conversational subsystem

- **Failure Points**
  - Access validation on requested documents
  - Retrieval returning no relevant results
  - First-pass retrieval succeeds but second-pass expansion fails
  - LLM synthesis fails after retrieval, causing fallback response
  - Lineage persistence can fail after generation attempt

- **Notes on Hidden Complexity**
  - This is a retrieval orchestration flow, not a thin wrapper over search
  - Confidence scoring changes prompting and response wording
  - Multi-hop retrieval is conditional and mode-dependent
  - It has its own conversation ID and lineage model, separate from standard chat sessions

- **Flow Name**
  Workflow Execution with Suspension and Resume

- **Trigger / Entry Point**
  - `POST /api/workflows/:workflowId/execute` in [`workflows.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/workflows.ts#L446)

- **Sequence of Components Involved**
  - Authenticated route validates workflow existence/access and request body
  - Creates `workflowExecutions` row with status `running`
  - Enqueues `execute-workflow` on `workflowExecutionQueue`, or falls back to in-process execution
  - `runWorkflowExecution()` in [`workflow-runner.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-runner.ts#L10) loads workflow definition and BYOK keys, then delegates to `workflowExecutor.execute()`
  - `workflowExecutor` walks node graph, runs ready nodes in parallel, and handles node types like `llm`, `rag`, `transform`, `media_ingest`, `transcribe`, `media_insights`
  - Async media nodes return suspension sentinels; runner persists `checkpoint`, `resumeTrigger`, `executionLog`, and marks execution `suspended`
  - Later, media worker completes and calls `resumeWorkflowExecution()` in [`workflow-runner.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-runner.ts#L117)
  - Resume path restores checkpoint, injects async outputs, and continues execution until complete/fail/suspend again

- **State Changes**
  - `workflowExecutions` row created
  - Status transitions: `running` → `suspended` → `running` → `completed`/`failed`/`cancelled`
  - `checkpoint`, `resumeTrigger`, `executionLog`, `output`, and `error` are updated over time

- **Failure Points**
  - Workflow missing or inaccessible
  - Queue unavailable
  - Node-level failures abort the workflow
  - Resume race handled through optimistic lock, but retries still add complexity
  - Suspended execution can fail later if checkpoint/workflow state no longer aligns

- **Notes on Hidden Complexity**
  - This is the most stateful flow in the system
  - Execution durability depends on DB state, queue delivery, and storage-backed async media work all staying coherent
  - The API responds `202` before the actual workflow logic finishes, so user-visible success is decoupled from execution success

- **Flow Name**
  Media Intelligence Pipeline

- **Trigger / Entry Point**
  - Starts from media file upload in `POST /api/files/upload`
  - Continues through `media-processing` jobs in [`queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts#L680)

- **Sequence of Components Involved**
  - Upload route detects media, stores original media through `storageProvider`, updates `files`, and enqueues `extract-metadata`
  - `extract_metadata` downloads stored media, extracts technical metadata, updates `files.metadata`, then enqueues `extract_audio`
  - `extract_audio` downloads media, extracts audio with FFmpeg helpers, uploads derived audio, transcribes through external transcription provider, enriches speakers, persists `transcriptSegments`, updates `files.metadata`, then enqueues `extract_frames`
  - `extract_frames` optionally extracts frames, runs vision analysis, persists `frameAnalyses`, generates `mediaInsights`, derives media knowledge chunks, replaces `documentChunks`, and may enqueue `document-indexing`
  - If the media job was started by a suspended workflow, it resumes the workflow via `resumeWorkflowExecution()`

- **State Changes**
  - `files.metadata` accumulates media metadata, transcription info, speaker info, frame-analysis status, insight-generation status
  - `transcriptSegments`, `frameAnalyses`, `mediaInsights`, and `documentChunks` are inserted or replaced
  - Media assets gain derivative storage keys such as extracted audio
  - Resulting media becomes searchable knowledge after indexing

- **Failure Points**
  - Storage download/upload failures

  - FFmpeg/media extraction errors
  - Transcription provider failures
  - Vision analysis or insight-generation failures
  - Replacing `documentChunks` before successful downstream indexing can create transitional states

- **Notes on Hidden Complexity**
  - This is really a chained workflow system inside the queue layer
  - Media processing is not only enrichment; it converts audio/video into the same retrieval substrate used by RAG
  - It also doubles as a workflow-resume trigger source, so it is both content pipeline and orchestration dependency

These are the highest-value execution paths I found because they dominate user-facing behavior and backend state mutation. If you want, I can do a second pass that turns this into a dependency graph: `route -> middleware -> orchestrator/service -> tables/queues -> external APIs`.

Dependency & Infra Mapping Prompt

Analyze this codebase to map all important technical dependencies and infrastructure assumptions.

Identify:
1. Frameworks, libraries, SDKs, and their roles
2. Databases, caches, queues, storage systems
3. External APIs and integrations
4. Authentication and authorization dependencies
5. Deployment/runtime assumptions
6. Environment variables and configuration-driven behavior
7. Critical single points of failure

Return:
- Technology Stack Map
- Runtime Dependency Graph
- External Service Dependencies
- Config / Environment Dependence
- Operational Risks

Technology Stack Map

Backend application
- `Fastify` powers the API server and route registration in [`services/backend/src/index.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/index.ts).
- `@fastify/cors`, `@fastify/multipart`, `@fastify/rate-limit`, `@fastify/swagger`, and `@fastify/swagger-ui` provide CORS, uploads, throttling, and API docs in [`services/backend/package.json`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/package.json).
- `zod` is the primary request/config validation layer across routes and config, e.g. [`services/backend/src/config/config.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/config/config.ts).
- `drizzle-orm` plus `postgres` are the ORM and PostgreSQL driver in [`services/backend/src/database/connection.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/connection.ts).
- `pgvector` is used for vector columns/search in the schema in [`services/backend/src/database/schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts).
- `bullmq` and `ioredis` back async jobs and Redis connectivity in [`services/backend/src/services/queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts).
- `winston` and pino-compatible Fastify logging are both present; Fastify uses structured logging in-process, while utility logging wraps broader behavior.
- `prom-client` powers Prometheus metrics exposed by backend telemetry/metrics paths.
- AI SDKs:
  - `openai`
  - `@anthropic-ai/sdk`
  - `groq-sdk`

  - `cohere-ai`
  - Ollama is accessed over HTTP through configured base URL in [`services/backend/src/config/config.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/config/config.ts) and provider code in [`services/backend/src/services/llm-providers.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/llm-providers.ts)
- Auth/security libraries:
  - `jsonwebtoken`
  - `bcryptjs`
  - `otplib`
  - `qrcode`
  - `google-auth-library`
- Document/media processing libraries:
  - `pdf-parse`
  - `mammoth`
  - `xlsx`
  - `marked`
  - `fluent-ffmpeg`
  - `@ffmpeg-installer/ffmpeg`

Frontend
- `React 18` + `Vite` in [`services/frontend/package.json`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/package.json)
- `react-router-dom` for route structure
- `@tanstack/react-query` for server-state caching
- `zustand` for client-side app state
- `reactflow` for workflow builder UI
- `recharts` for charts/analytics UI
- `react-markdown`, `prismjs`, `lucide-react`, `react-hot-toast`
- `crypto-js` is used for client-side localStorage “encryption” in [`services/frontend/src/utils/storage.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/utils/storage.ts)

Landing page
- Separate Vite/React app with nearly the same frontend dependency family in [`services/landing-page/package.json`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/landing-page/package.json)

Runtime Dependency Graph

Primary runtime chain
- `Frontend` -> HTTP calls -> `Backend Fastify API`
- `Backend API` -> `PostgreSQL`
- `Backend API` -> `Redis`
- `Backend API` -> `BullMQ queues`

- `BullMQ workers` -> `PostgreSQL`
- `BullMQ workers` -> `Redis`
- `BullMQ workers` -> `Storage provider`
- `Backend + Workers` -> `LLM/embedding/transcription/payment/OAuth external APIs`

Backend internal dependency shape
- Routes depend directly on middleware, DB, services, repositories, and utilities.
- Application orchestrators depend on shared services and shared repositories.
- Queue workers depend on nearly every heavy subsystem: file processing, embeddings, workflow runner, storage, media processing, and DB in [`services/backend/src/services/queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts).

State systems
- `PostgreSQL` is the system of record for:
  - users/auth/security
  - chat sessions/messages
  - files/document chunks
  - workflows/executions
  - notebooks
  - bookmarks
  - preferences
  - usage and lineage
- `Redis` is used for:
  - queue transport
  - caching
  - token handoff for OAuth exchange
  - token blacklist when available
- `Storage provider` is used for:
  - original files
  - media derivatives
  - downloadable artifacts
  - optionally training-related exports/signals via the training storage path in [`services/backend/src/services/file-storage.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/file-storage.ts)

Operational mode changes
- If Redis is unavailable, queues degrade or disable and some flows fall back to synchronous behavior in [`services/backend/src/services/queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts).
- If pgvector or indexing flows are impaired, the system can still store/upload/process files but retrieval quality degrades.

External Service Dependencies

AI/model providers
- `OpenAI` for chat/LLM calls and likely embeddings fallback
- `Anthropic` for chat generation
- `Groq` for chat generation and also some media/transcription-related external calls
- `Cohere` for embeddings/search-related operations
- `Ollama` for self-hosted/local inference over HTTP

These are used through the provider abstraction in [`services/backend/src/services/llm-providers.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/llm-providers.ts) and embedding logic in related services.

Google
- `Google OAuth` for login in [`services/backend/src/services/googleAuthService.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/googleAuthService.ts)
- `Google Workspace OAuth` integration in [`services/backend/src/routes/integrations.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/integrations.ts)

Payments/subscriptions
- `Razorpay`
- `Lemon Squeezy`

These are called from subscription routes in [`services/backend/src/routes/subscriptions.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/subscriptions.ts).

Storage backends
- `Cloudflare R2`
- `AWS S3`
- local/volume-backed filesystem

Configured in [`services/backend/src/services/file-storage.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/file-storage.ts) and [`services/backend/src/services/local-file-storage.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/local-file-storage.ts)

Media/tooling assumptions
- FFmpeg must be runnable for media extraction, audio processing, and rendering.
- Video/audio analysis depends on temp filesystem access and external model services.

Config / Environment Dependence

Backend config surface

- Centralized in [`services/backend/src/config/config.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/config/config.ts)
- Core env-controlled behavior includes:
  - `PORT`, `NODE_ENV`, `CORS_ORIGIN`
  - `DATABASE_URL`
  - `REDIS_URL`
  - `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GROQ_API_KEY`, `COHERE_API_KEY`, `OLLAMA_API_KEY`, `OLLAMA_BASE_URL`
  - `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `GOOGLE_REDIRECT_URI`
  - `JWT_SECRET`, `JWT_EXPIRES_IN`
  - `API_KEY`
  - `ENCRYPTION_KEY`
  - `STORAGE_PROVIDER`, `STORAGE_PATH`, `VOLUME_MOUNT_PATH`
  - `R2_*` credentials and bucket names

Frontend config surface
- Defined in [`services/frontend/src/config/env.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/config/env.ts)
- Important frontend env vars:
  - `VITE_API_URL`
  - `VITE_API_BASE_URL`
  - `VITE_GOOGLE_REDIRECT_URI`
  - dev-only auth bootstrap vars such as `VITE_AUTO_LOGIN`, `VITE_DEV_AUTH_TOKEN`, `VITE_DEV_AUTH_USER_JSON` used in [`services/frontend/src/contexts/AuthContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/AuthContext.tsx)

Configuration-driven behavior that materially changes runtime
- Redis available vs unavailable changes queue/background behavior.
- Storage provider setting changes download/upload semantics.
- Production mode triggers stricter secret validation in [`services/backend/src/config/config.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/config/config.ts).
- BYOK user API keys alter which external provider credentials are used.
- Frontend API base URL determines whether it talks directly to backend or via some external ingress.

Deployment assumptions
- Node 18+ and npm workspaces from root [`package.json`](/Users/aeishwary/JoaLLM-platform/joallm-platform/package.json)
- PostgreSQL 15 with pgvector
- Redis 7
- Persistent volume or object storage for file retention
- Queue worker as a separate process for full async functionality
- Railway/private-network assumptions appear in DB comments and deployment docs; SSL is explicitly disabled for internal DB in

[`services/backend/src/database/connection.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/connection.ts)
- Monitoring assumes Prometheus/Grafana stack, visible in compose and monitoring config

---

Operational Risks

Critical single points of failure
- `PostgreSQL` is the biggest SPOF.
  - Almost every meaningful user flow depends on it.
  - If DB is down, auth, chat history, file state, workflows, security state, and retrieval all break or degrade severely.
- `Backend API process` is a SPOF for all frontend behavior.
  - There is no active service mesh or gateway abstraction in the actual code path.
- `Shared schema ownership` is a structural SPOF.
  - Many modules query the same tables directly, so schema issues ripple system-wide.
- `Redis` is a conditional SPOF.
  - Not everything dies without it, but background processing, OAuth token exchange, caching, and some token-revocation behavior are impaired.
- `Queue worker` is a practical SPOF for nontrivial document/media/workflow workloads.
  - The API may still respond, but core platform promises degrade.

Provider/API dependence risks
- AI providers are multi-vendor, which is good, but many core flows still require at least one provider to be healthy.
- Media and RAG quality depend heavily on external inference/embedding/transcription availability and quotas.
- Payment flows depend on third-party APIs and webhook integrity.
- Google auth/integrations depend on correct OAuth redirect and secret configuration.

Infrastructure fragility signals
- Queue fallback behavior means the same feature can run in very different modes depending on Redis health.
- Local volume storage and cloud storage are both supported, which increases branch complexity in file handling.
- The frontend depends on backend URL correctness; docs in the repo show past drift around ports/base URLs.
- There are committed env/log artifacts in the tree, which is an operational hygiene warning sign.

Auth/security dependency risks
- JWT, refresh-token, blacklist, OAuth, 2FA, and role projection are spread across multiple modules; failure or inconsistency in one part affects many flows.
- Client-side token persistence in localStorage makes frontend compromise especially damaging.
- Access control is not only middleware-driven; some flows also enforce ownership in-route, increasing inconsistency risk.

Net assessment
- The platform is highly dependent on a stable `DB + backend API + Redis/worker + at least one AI provider` core.
- The architecture has graceful degradation in some places, but the important product promises depend on several services being healthy at once.
- Operationally, PostgreSQL and the backend process are the true hard dependencies; Redis and the worker are the major “quality/functionality multipliers”; external AI/storage/payment providers are the major ecosystem dependencies.

Data Model Understanding Prompt

Act as a senior backend architect reviewing the data model.

Analyze the codebase and explain:
1. Core entities and their relationships
2. Which entities are central to system behavior
3. Where state is stored and mutated
4. Where schema design seems strong vs weak
5. Missing constraints, weak relations, or duplication risks
6. Any signs of orphan data, invalid states, or hidden coupling

Return:
- Core Data Entities
- Relationship Map
- State Ownership
- Mutation Hotspots
- Data Integrity Risks
- Recommendations

Core Data Entities

The system has a few true core aggregates, and everything else hangs off them.

- `users` is the root identity record. It carries both account identity and product state like `role`, `subscriptionTier`, `usageStats`, and some API-key-related data in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L16).
- `chatSessions` and `messages` form the main conversational model in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L86) and [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L104).
- `files` and `documentChunks` are the knowledge substrate for uploads, RAG, media analysis, and workflow inputs in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L125) and [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L214).
- `workflows` and `workflowExecutions` model user-authored automations plus durable execution state in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L238) and [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L256).
- `notebooks` and `notebookCells` are the exploratory workspace layer in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L557) and [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L568).

- `userPreferences`, `userSecurity`, and `trainingConsent` are one-row-per-user side tables that hold long-lived account state in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L500), [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L524), and [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L622).
- `apiUsage`, `inferenceRuns`, `searchHistory`, `ragSearchSessions`, `messageFeedback`, and `messageRagSources` are telemetry/lineage tables. They are not root entities, but they matter because they record how AI outputs were produced and used.

The most central entities to actual system behavior are `users`, `files`, `documentChunks`, `chatSessions`, `messages`, and `workflowExecutions`. Those are the ones multiple subsystems depend on directly.

Relationship Map

The strongest ownership chains are good and easy to follow:

- `users -> chatSessions -> messages`
- `users -> files -> documentChunks`
- `users -> workflows -> workflowExecutions`
- `users -> notebooks -> notebookCells`
- `users -> userPreferences`
- `users -> userSecurity`
- `users -> trainingConsent`
- `messages -> messageFeedback`
- `messages -> messageRagSources -> documentChunks`

Those are real relationships with foreign keys and mostly sensible `onDelete` behavior in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts).

The weaker relationships are where the model gets fuzzy:

- `bookmarks.itemId` is polymorphic and has no foreign key to the target object in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L481).
- `notebookCells.attachedDocuments` is a `jsonb` string array, not a join table, so document attachment integrity is not enforced in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L579).
- `workflows.nodes` and `workflows.edges` are stored as raw `jsonb`; any references to files, models, or other resources live inside untyped node payloads in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L244).

- `searchHistory.fileIds` and `ragSearchSessions.documentIds` are arrays/JSON rather than relational links.
- `messages.sessionId` is nullable, and the code actively uses that to create lineage-only synthetic assistant messages in [`rag-lineage-repository.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/repositories/rag-lineage-repository.ts#L114). That means `messages` is not purely “chat messages in a session” anymore.

The result is a mixed model: core ownership is relational, but cross-feature references often escape into JSON.

State Ownership

State is not owned by one layer. It is spread across API routes, orchestrators, and workers.

- Chat state is created and mutated in both [`chat-orchestrator.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/application/chat/chat-orchestrator.ts#L148) and [`chat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts#L846).
- File state starts in [`files.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts#L532) but most lifecycle mutation happens later in [`queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts#L314).
- Workflow execution state is persisted and resumed through [`workflow-runner.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-runner.ts#L10) and [`workflow-runner.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-runner.ts#L117).
- Preference and security state is lazily created and updated from route handlers in [`preferences.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/preferences.ts) and [`security.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/security.ts).
- RAG lineage writes happen through a repository that also inserts synthetic `messages`, which creates hidden coupling between retrieval and chat history semantics in [`rag-lineage-repository.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/repositories/rag-lineage-repository.ts).

The biggest mutation hotspots are:

- [`services/backend/src/services/queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts)
- [`services/backend/src/routes/chat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts)

-
[`services/backend/src/routes/files.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts)
-
[`services/backend/src/services/workflow-runner.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-runner.ts)

That split ownership is the real architectural story: the database is the shared state machine, and many actors mutate the same entities at different stages.

Data Integrity Risks

The schema is strong where it models obvious parent-child ownership. It is weak where product flexibility won over relational integrity.

Strong areas:
- Good use of unique constraints for identity and one-row-per-user side tables.
- Solid FK/cascade design on core chains like user -> session -> message and user -> file -> chunk.
- `workflowExecutions` explicitly stores checkpoint/resume state, which is better than hiding runtime state in memory.

Weak areas:
- Heavy `jsonb` usage for behaviorally important state: workflow graphs, execution logs, checkpoints, file processing metadata, active sessions, notebook doc attachments, model settings. That makes invalid combinations easy and database guarantees weak.
- Status duplication is everywhere. `files.status` coexists with metadata like `processingStage`, `indexingStatus`, `vectorSearchAvailable`, `keywordSearchAvailable`. Those can drift.
- `messages` is overloaded. It stores normal session chat and also lineage-only synthetic RAG output with `sessionId = null`.
- The DB enum for notebook cells allows `chart`, `agent`, and `debug`, but the route validator only accepts `markdown`, `code`, `ai`, and `knowledge` in [`notebooks.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/notebooks.ts#L21). That is schema/application drift.
- `bookmarks` can easily point at deleted or nonexistent objects because the target is polymorphic and unenforced.
- `documentChunks.fileId` is not declared `notNull()`, even though the code treats chunks as belonging to files.
- `userSecurity.activeSessions` is stored as JSON, while token revocation also exists separately in `revokedTokens`. That is duplicate session truth.
- The code writes audit logs through raw SQL in [`audit.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/utils/audit.ts), but `audit_logs` is not represented in the main Drizzle schema. That suggests partial schema ownership outside the typed model.

Signs of orphan data or invalid state risk:
- bookmarks to deleted targets

- notebook attached document IDs that no longer exist
- workflow node payloads referring to files/resources that are deleted or unauthorized
- suspended workflow executions with stale checkpoints
- files marked `processed` while vector/indexing metadata says otherwise
- messages with no session being treated like chat records by downstream consumers

Recommendations

1. Tighten the relational core before adding more features.
Use real join tables for high-value cross-links:
- notebook cell -> document
- search session -> document
- bookmarks for the few target types that matter most

2. Reduce JSON as a source of truth.
Keep JSON for flexible metadata, but normalize fields that drive behavior:
- file indexing state
- active sessions
- workflow resource references
- notebook attached documents

3. Add consistency constraints.
Introduce DB checks or service-level invariants for:
- `files.status` vs processing/indexing fields
- `workflowExecutions.status` vs `checkpoint` / `resumeTrigger` / `output`
- `messageFeedback` field combinations by feedback type

4. Decide what `messages` means.
If it is a universal inference log, name/model it that way. If it is chat-only, stop storing lineage-only RAG outputs there and give lineage its own response table.

5. Eliminate schema drift.
Align route validators with DB enums, and bring raw-managed tables like `audit_logs` into the declared schema or clearly document them as external.

6. Make ownership explicit in async flows.
The queue and workflow runner mutate core entities heavily. Wrap critical multi-step transitions in clearer state transition helpers so file/workflow state cannot drift across route and worker codepaths.

Overall, the model has a solid relational backbone, but too much cross-feature behavior leaks into JSON and duplicated status fields. The main risk is not missing tables; it is losing a single authoritative truth for state as the platform keeps growing.

Business Logic Mapping Prompt

Analyze the codebase and extract the true business logic of the system.

Identify:
1. Core business rules enforced in code
2. Hidden business rules scattered across modules
3. Validation rules
4. Workflow/state transition rules
5. Permission-related logic
6. Rules enforced only in frontend or only in backend
7. Inconsistencies between intended behavior and implementation

Return:
- Explicit Business Rules
- Implicit Business Rules
- State Transition Logic
- Validation Logic
- Permission Logic
- Inconsistencies / Gaps
Explicit Business Rules

These are rules the backend clearly enforces as product behavior, not just data shape.

- Users have tiered access. Model access and some features are gated by subscription tier and role, via access-control logic and route prehandlers such as `requireModelForTier()` in [`chat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts#L847) and tier/permission projection in [`access-control.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/access-control.ts).
- Authenticated ownership is the default rule for user-created assets like chat sessions, messages, files, workflows, notebooks, bookmarks, and integrations.
- A chat message is persisted before model generation. That means “user sent message” is a committed business event even if the LLM later fails in [`chat-orchestrator.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/application/chat/chat-orchestrator.ts#L148).
- File upload creates a durable file record first, then processing and indexing happen later. Upload success does not mean searchable success in [`files.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts#L532) and [`queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts#L314).
- Workflow execution is asynchronous and durable. An execution can be `running`, `suspended`, `completed`, `failed`, or `cancelled`, with checkpoint/resume semantics in [`workflow-runner.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-runner.ts#L10).
- Feedback is one-per-user-per-message through a unique constraint and upsert behavior in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L600) and feedback routes.

- Some “system” account behavior is hardcoded by email and role projection. That is real business logic, even if it looks like an operational shortcut, in [`access-control.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/access-control.ts#L31) and auth hydration in [`middleware/auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/middleware/auth.ts#L226).

Implicit Business Rules

These are rules the system depends on, but they are scattered rather than declared centrally.

- `files` plus `documentChunks` are the canonical knowledge substrate for multiple products. RAG, media analysis, workflows, and some chat features all assume uploaded knowledge ends up there.
- A “processed” file is not the same thing as a fully indexed file. The code treats processing, keyword search readiness, and vector search readiness as overlapping but distinct states via `files.status` and `files.metadata`.
- Chat sessions have hidden lifecycle semantics: `isActive`, `completionStatus`, `turnCount`, auto-title generation, and regeneration/copy flags on messages all contribute to how a conversation is treated.
- Messages are not purely chat records. The RAG lineage layer inserts assistant messages with `sessionId: null`, so `messages` is functioning partly as a generic AI output ledger in [`rag-lineage-repository.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/repositories/rag-lineage-repository.ts#L114).
- Public workflows are intended to be shareable/executable, but workflow node payloads can still carry private resource references. That creates an implicit and unsafe business rule: “public definition, private dependencies.”
- One-row-per-user tables like preferences, security, and training consent are lazily created on demand. The real rule is “absence means default state,” not “row must exist from account creation.”

State Transition Logic

The clearest state machines in the system are files, workflows, auth/security, and conversational persistence.

- **Files**
  - `uploaded` -> `processing` -> `processed` or `failed`
  - Additional hidden sub-states live in metadata such as `processingStage`, `indexingStatus`, transcript/frame/insight flags, and search-availability flags.
  - Media files have chained transitions across metadata extraction, audio extraction/transcription, frame analysis, insight generation, and optional indexing in [`queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts#L680).

- **Workflow executions**
  - `running` -> `completed`

  - `running` -> `failed`
  - `running` -> `suspended` -> `running` -> terminal state
  - Resume depends on persisted checkpoint plus async trigger payload in [`workflow-runner.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-runner.ts#L117).

- **Authentication**
  - Login can produce a temporary `pending_2fa` auth state before full session issuance.
  - Failed logins increment counters and can lock accounts via `userSecurity`.
  - Refresh produces new access based on a valid refresh token, but revocation rules are weaker than they should be.

- **Chat**
  - Session may be created on first message.
  - User message is inserted before inference.
  - Assistant message is inserted after inference success.
  - Title generation is a secondary async-ish side effect, not part of the core transaction.

Validation Logic

Validation is strongest at the request-schema level and weaker at the cross-entity consistency level.

- Most route bodies use `zod` schemas for shape/type validation, especially in auth, chat, RAG, and workflows.
- File upload validates file size/type and distinguishes media/document handling in [`files.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts).
- Auth validates email/password formats and 2FA payloads.
- Chat validates message payload, model selection, and sometimes selected document IDs.
- RAG validates search/chat payloads, but document ownership checks are conditional and stronger when explicit file/document IDs are supplied than when they are omitted.
- Some validation is enum-driven in DB but narrower in route code. Example: notebook cell types supported by API do not match the broader DB enum in [`notebooks.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/notebooks.ts#L21) vs [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L568).

A recurring pattern is: request shape is validated well, but semantic invariants across tables are often not.

Permission Logic

Permission logic exists in both central and distributed forms.

- Centralized permission/tier logic lives in the access-control service and auth middleware.

- Routes commonly use `authenticateToken` plus specific permission checks such as `requirePermission('knowledge.write')` on file upload.
- Many routes also perform manual ownership checks against DB rows, especially for files, messages, workflows, and chat sessions.
- Public workflows are an exception path that intentionally bypasses strict owner-only execution.
- RAG uses `optionalAuth` on some routes, which creates materially different access behavior depending on whether the caller is authenticated.
- Frontend also gates features based on the `/api/me/access` snapshot, but backend enforcement is the true authority for sensitive operations.

The real permission model is therefore:
- role/tier grants capability
- row ownership grants object access
- some routes add ad hoc exceptions
- some frontend behavior is hidden/disabled based on access snapshot even when not every backend edge is uniformly enforced

Inconsistencies / Gaps

- The intended enterprise/workspace model is only partially real. The schema has organizations/workspaces/memberships, but much of the implementation is still primarily user-scoped.
- RAG access behavior is inconsistent: explicit document selection gets more scrutiny than implicit/global search.
- The notebook product surface and the DB schema disagree on supported cell types.
- Messages are treated as both conversational records and generic lineage outputs, which blurs business meaning.
- File-processing state is duplicated across top-level status and metadata flags, so the business rule for “ready” is not singular.
- Security/business policy includes hardcoded identity shortcuts based on email address, which is implementation-specific logic masquerading as product policy.
- Frontend and backend do not always enforce the same rules. The frontend hides or shapes access by entitlement snapshot, but backend object/resource validation is uneven across modules.
- Some important business rules are effectively encoded in async workers rather than synchronous APIs. For example, “what counts as a processed media file” is defined by queue behavior, not a clean domain policy object.

Overall, the codebase enforces a real business model: tiered AI workspace, user-owned knowledge assets, durable workflow execution, and multi-surface AI usage tracking. But the rules are spread across middleware, routes, orchestrators, and workers, and several of the most important ones are implicit, duplicated, or inconsistent depending on execution path.

API Surface Analysis Prompt

Act as an API architect reviewing this codebase.

Analyze all exposed APIs and determine:
1. Main endpoints and their purpose
2. Request/response consistency
3. Validation quality
4. Authentication/authorization handling
5. Error handling patterns
6. Idempotency concerns
7. Versioning strategy or lack of it
8. Endpoints that are risky, overly permissive, or fragile

Return:
- API Inventory
- Endpoint Categories
- Validation & Security Review
- Error Handling Patterns
- Consistency Issues
- High-Risk Endpoints

API Inventory

The backend exposes one broad application API under `/api`, with a few major endpoint families:

- `auth`
  - register, login, refresh, logout, Google OAuth callback/exchange, password reset, email verification-like flows, session bootstrap
  - Purpose: identity, session issuance, recovery, federated auth
- `security`
  - 2FA setup/verify/disable, session/security settings
  - Purpose: account hardening and login-step enforcement
- `chat`
  - send, stream, session CRUD, message actions, exports, save-to-knowledge, feedback-like message signals
  - Purpose: conversational AI, session management, downstream knowledge capture
- `files`
  - upload, list, get, delete, processing status, transcript/media-related accessors
  - Purpose: document/media ingestion and knowledge asset management
- `rag`
  - search, chat, sessions/query logging, analytics-like endpoints
  - Purpose: retrieval, grounded answering, search history
- `workflows`
  - create/update/delete/list/get/execute/cancel/share-like operations
  - Purpose: user-authored automation graphs and execution state
- `notebooks`
  - notebook and cell CRUD/execution-oriented endpoints
  - Purpose: structured AI workspace

- `preferences`, `user-settings`, `me`
  - access bootstrap, preferences, exports
  - Purpose: personalization and entitlement hydration
- `bookmarks`
  - create/list/delete bookmarks across different item types
- `feedback`
  - message feedback and training-consent style endpoints
- `subscriptions`
  - plan/payment/webhook-oriented behavior
- `integrations`
  - external provider OAuth and token management, at least Google Workspace

The API is product-wide rather than bounded by independent service contracts. It behaves like one app API, not a set of separately versioned domain APIs.

Endpoint Categories

The endpoints fall into a few contract styles.

- **Synchronous CRUD**
  - auth profile/preferences/bookmarks/notebooks/workflow definitions
  - Generally straightforward request/response JSON
- **Transactional AI calls**
  - `/api/chat/send`, `/api/rag/chat`, some notebook/AI execution paths
  - Request triggers persistence plus provider call plus telemetry
- **Streaming**
  - `/api/chat/stream`
  - Uses SSE-style raw response handling, which diverges from normal JSON patterns
- **Async job submission**
  - file upload and workflow execution
  - Client often gets an accepted or partial-success response before background completion
- **Operational/analytics side effects**
  - feedback, search history, usage, lineage
  - These often piggyback on primary user flows instead of being isolated subsystems

This mix is normal for an AI platform, but it increases inconsistency risk because the contract semantics differ a lot by endpoint family.

Validation & Security Review

Validation quality is decent at the shape level and uneven at the policy level.

- `zod` is used broadly for request body validation in routes such as auth, chat, and RAG, which is a strong baseline.
- File uploads validate MIME/extension/size and branch by media/document type in [`files.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts).
- Auth routes validate login/register/reset payloads and 2FA challenge formats.

- Chat and RAG payloads are structurally validated, but semantic validation across related resources is inconsistent.
- Some route validators are out of sync with the schema, such as notebook cell types in [`notebooks.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/notebooks.ts#L21) versus the DB enum in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L568).

Auth/authz handling is mixed: strong in some places, ad hoc in others.

- Most sensitive endpoints use `authenticateToken`.
- Many also use capability guards like `requirePermission(...)` or model-tier checks.
- Object-level authorization is often reimplemented inside handlers by checking row ownership manually.
- Some endpoints use `optionalAuth`, especially in RAG, which materially changes the access model of those routes.
- Frontend entitlement gating exists, but backend enforcement remains inconsistent across modules.

Net assessment:
- Request validation: `moderately strong`
- Authorization consistency: `fragile`
- Object-level access control: `uneven`

Error Handling Patterns

There is a recognizable but not fully uniform pattern.

- Many routes use `try/catch` and return JSON error objects with an error message and HTTP code.
- Validation failures usually become 400-ish responses.
- Auth failures are typically 401/403.
- Missing resources usually map to 404.
- Background-job workflows often return success/accepted before later failure is known, especially file processing and workflow execution.
- Streaming chat has a separate error pattern because part of the response may already be sent before persistence finishes.
- Some secondary side effects fail softly:
  - auto-title generation
  - usage logging
  - lineage recording
  - cache writes

This means user-visible responses are often “best effort around the primary path,” with telemetry and enrichment treated as non-blocking. That’s reasonable, but not always explicit in the contract.

Consistency Issues

The API surface is not chaotic, but it is not fully consistent either.

- No obvious global response envelope. Some endpoints return direct objects, some return `{ success, ... }`, some return arrays, and streaming bypasses normal JSON entirely.
- Sync and async endpoints are mixed without a uniform job contract. File upload and workflow execution both create long-running work, but they do not present one common async resource model.
- Ownership enforcement is route-specific rather than uniformly mediated.
- Some feature families use richer typed contracts than others; auth/chat/RAG are more structured than bookmarks/settings/integrations.
- Versioning is effectively absent. The API seems to rely on in-place evolution under the same `/api` namespace, with compatibility handled informally.
- Some business semantics are encoded in metadata-heavy fields rather than surfaced as first-class API states, especially file readiness and workflow progress.
- Streaming chat duplicates logic from normal chat instead of cleanly sharing one contract core, which increases drift risk.

High-Risk Endpoints

These are the endpoints or endpoint families that appear most risky, overly permissive, or operationally fragile.

- **`/api/rag/search` and `/api/rag/chat`**
  - Risk: permissive auth model via `optionalAuth`; access behavior changes significantly depending on whether file IDs are explicitly supplied.
  - Fragility: retrieval and authorization semantics are not uniform.
- **`/api/workflows/:id/execute`**
  - Risk: public execution plus node payload references can create cross-resource access problems.
  - Fragility: async, stateful, checkpointed execution with many side effects and resume paths.
- **`/api/files/upload`**
  - Risk: large multipart ingestion, immediate persistence plus deferred processing, queue/storage dependency, memory pressure.
  - Fragility: accepted upload does not guarantee finished processing/indexing.
- **`/api/chat/stream`**
  - Risk: partial output can be sent before final DB state is committed.
  - Fragility: separate implementation path from normal chat increases divergence.
- **OAuth/integration callback endpoints**
  - Risk: state handling and token storage are security sensitive.
  - Fragility: success depends on redirect config, Redis handoff, provider callback correctness.
- **User export/settings endpoints**
  - Risk: potential overexposure of internal or security-sensitive fields if DTO filtering is weak.
- **Bookmarks and other polymorphic-reference endpoints**
  - Risk: accept references the DB cannot validate; easy source of stale or misleading state.

Versioning Strategy or Lack of It

There is no strong API versioning strategy visible in the active code. The API appears to be:

- one evolving `/api` surface
- no explicit `/v1`, `/v2` split
- no visible compatibility layer or deprecation policy
- contract changes likely coordinated by codebase coupling rather than versioned interface management

That is workable for an internal monorepo-style product, but it becomes risky once third-party clients, mobile apps, or multiple frontend versions need stability.

Overall, the API is broad and functional, with decent local validation and many purposeful endpoint families. The weakest parts are contract consistency, object-level authorization uniformity, async job semantics, and the lack of clear versioning. If I were hardening it, I'd prioritize a common error/response shape, stricter centralized authorization rules, and a unified long-running-job contract for uploads and workflow execution.

Async / Queue / Worker Analysis Prompt

Async / Queue / Worker Analysis Prompt

Analyze all asynchronous execution in this codebase.

Identify:
1. Queues, workers, background jobs, schedulers
2. What tasks are executed asynchronously
3. Job lifecycle and status tracking
4. Retry logic, backoff, and failure recovery
5. Duplicate execution risks
6. Orphan job or stuck-state risks
7. How async state is communicated back to users

Return:
- Async Components
- Job Lifecycle
- Reliability Mechanisms
- Failure Modes
- User Visibility of Async State
- Recommendations

Async Components

The async system is centered on `BullMQ + Redis + a dedicated worker process`, not on cron-style schedulers.

- The main worker entrypoint is [`worker.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/worker.ts).
- Queue orchestration lives in [`queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts).
- Workflow suspension/resume logic is coordinated through [`workflow-runner.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-runner.ts).

The important async job families are:

- `document-processing`
  - extract text, create chunks, update file processing metadata
- `document-indexing`
  - generate embeddings and make documents searchable for RAG
- `media-processing`
  - metadata extraction, audio extraction, transcription, frame analysis, insight generation
- `workflowExecutionQueue`
  - run workflow graphs asynchronously
- workflow resume triggers

- not a separate product-facing queue concept, but async media jobs can resume suspended workflow executions

I did not see a meaningful scheduler/cron subsystem driving periodic jobs. The async model is mostly event-driven off user actions.

Job Lifecycle

The lifecycle is not managed by a universal job abstraction. It is split between queue state and domain-table state.

Files
- Upload creates a `files` row immediately in [`files.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts).
- The route then enqueues work if Redis/queues are available.
- Workers update `files.status` plus `files.metadata.processingStage`, indexing flags, transcript/frame/insight flags in [`queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts).
- Derived rows such as `documentChunks`, `transcriptSegments`, `frameAnalyses`, and `mediaInsights` are inserted later.

So the real lifecycle is:
- API creates durable record
- queue job advances state
- user polls/read endpoints to observe eventual completion

Workflow executions
- Execute endpoint creates a `workflowExecutions` row before any real work finishes.
- Queue job or in-process fallback runs the workflow.
- Execution state is tracked in the DB: `running`, `suspended`, `completed`, `failed`, `cancelled`.
- Suspension persists `checkpoint`, `resumeTrigger`, and `executionLog`.
- Later async completion resumes from persisted checkpoint in [`workflow-runner.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-runner.ts#L117).

This is the cleanest explicit async state machine in the system.

Reliability Mechanisms

There are some real reliability measures, but they are uneven.

- Using BullMQ gives job persistence, worker separation, and queue retry primitives.
- The worker process is separate from the API process, which helps keep heavy tasks out of request latency.

- Workflow executions persist checkpoints and can resume after async dependencies complete. That is a strong design choice.
- Some resume/update paths use optimistic locking logic to reduce double-resume races in workflow execution.
- Queue availability is checked, and some paths fall back to synchronous or degraded behavior if Redis is unavailable.

What appears weaker:

- There is no single domain-level “job” table abstracting all async work.
- File/media jobs rely heavily on mutable metadata fields as ad hoc state tracking.
- Retry/backoff behavior exists at the queue framework level, but the domain logic does not always look idempotent enough to make retries safe by default.
- Recovery from mid-pipeline partial completion often depends on overwriting or recomputing downstream state rather than a formal compensating strategy.

Failure Modes

The main async failure risks are state drift, duplicate effects, and incomplete visibility.

- **Partial completion**
  - A file row may exist while chunking/indexing failed.
  - A media file may have transcript data but no frames, or frames but no reindexed chunks.
- **Stuck status**
  - `files.status` and metadata sub-states can become inconsistent if one stage fails after another stage partially succeeded.
  - A workflow execution can remain `suspended` if the expected resume trigger never arrives.
- **Duplicate execution**
  - Retries or repeated submissions can recreate chunks, transcript segments, insights, or workflow side effects unless the code explicitly replaces or deduplicates them.
  - Some media/chunk flows do delete-and-recreate behavior, which helps, but it is not uniformly formalized.
- **Queue/API split-brain**
  - The API may report accepted/success while the queue later fails.
  - If Redis is down and the code falls back, behavior changes materially between environments.
- **Orphan derived data**
  - If parent cleanup is incomplete or if a pipeline is interrupted between delete/recreate phases, derived rows can become stale.
- **Mid-stream/mid-job visibility mismatch**
  - Users can observe a record before the system has reached a coherent final state.

Duplicate execution risks
- Workflow execution retries are the highest-risk area because node outputs may not all be naturally idempotent.
- Media processing has chained jobs; duplicate enqueues can repeat expensive extraction or regenerate derivative artifacts.

- File indexing can rerun after chunk insertion, which is safer than duplicating ingestion from scratch, but still relies on replacement/update discipline.

Orphan or stuck-state risks
- `workflowExecutions.status = suspended` with stale `checkpoint` or never-fired `resumeTrigger`
- `files.status = processing` indefinitely if a queue worker dies mid-stage
- metadata flags suggesting search/transcript readiness while derived tables are incomplete
- accepted uploads with no downstream worker consumption if Redis/worker topology is unhealthy

User Visibility of Async State

User visibility exists, but it is fragmented and mostly polling-oriented.

- Files expose status through the `files` table and related endpoints.
- Workflow executions are first-class rows, so users can query execution status/log/output later.
- Chat is mostly synchronous from the user's perspective, except streaming.
- There is no obvious unified "job center" or cross-domain async event model.
- Async completion is communicated mainly through:
  - reading updated resource state
  - checking workflow execution records
  - later seeing derived artifacts become available

This means the user experience depends on each product area surfacing its own async state. There is no shared platform abstraction for "your job is queued/running/retrying/stuck."

Recommendations

1. Add a platform-level async job model.
Keep BullMQ, but introduce a small DB-backed job registry so uploads, indexing, media tasks, and workflow runs share a consistent lifecycle and user-facing status model.

2. Make pipeline stages explicitly idempotent.
Every async handler should be safe to rerun. Prefer upsert/replace semantics with deterministic keys over append-only writes where possible.

3. Tighten stuck-state recovery.
Add watchdog/reaper logic for:
- files stuck in `processing`
- executions stuck in `running` or `suspended`
- jobs whose queue state and DB state diverge

4. Separate top-level status from stage detail more cleanly.
Use one authoritative coarse status plus a structured stage history, instead of many overlapping metadata booleans that can drift.

5. Expose async progress uniformly to clients.
Uploads and workflow executions should share a more consistent contract:
- accepted
- queued
- running
- retrying
- blocked
- completed
- failed

6. Add clearer retry policy per job type.
Differentiate between transient external failures, deterministic parsing failures, and user-data errors. Right now queue-level retries likely do too much of the work without enough domain-specific classification.

7. Protect against duplicate enqueue.
Use dedupe keys or state guards for jobs that should only have one active instance per file/execution/stage.

Overall, the async architecture is real and fairly capable, especially around workflow durability. The weak point is not “lack of queues”; it is that async state is tracked differently in each subsystem, which increases drift, retries-with-side-effects, and user confusion when work stalls.

Auth / Security Surface Mapping Prompt

Auth / Security Surface Mapping Prompt

Act as a security-minded architect.

Map the full authentication and authorization model in this codebase.

Identify:
1. Login/auth flows
2. Token/session handling
3. Role/permission enforcement
4. Backend vs frontend enforcement gaps
5. Sensitive routes and operations
6. Secrets handling patterns
7. Security assumptions that may be unsafe

Return:
- Auth Model
- Authorization Model
- Enforcement Locations
- Sensitive Operations
- Weak Security Signals
- Immediate Investigation Areas

Auth Model

The system uses `JWT-based stateless auth with partial server-side session overlays`.

Login/auth flows:
- Email/password login and registration live in [`auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts).
- Google OAuth login is handled through callback/exchange logic in [`auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/auth.ts#L584) and [`googleAuthService.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/googleAuthService.ts).
- 2FA is modeled as an intermediate authenticated state using a `pending_2fa` token, then upgraded through verification in [`security.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/security.ts#L322).

Token/session handling:
- Access and refresh tokens are JWTs generated in auth middleware/helpers in [`middleware/auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/middleware/auth.ts).
- Refresh is verified cryptographically and used to mint new access without a strong server-side session record.

- Logout and revocation are only partially stateful through `revokedTokens` and blacklist-style logic.
- `userSecurity.activeSessions` stores session-like state in JSON in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L524), but it is not the sole authority for token validity.
- On the frontend, tokens are stored in browser storage through [`AuthContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/AuthContext.tsx) and wrapped by reversible client-side storage logic in [`storage.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/utils/storage.ts).

The practical model is: bearer JWTs are the primary trust artifact, with some auxiliary DB/Redis/session metadata around them.

Authorization Model

Authorization is a hybrid of `role/tier capability checks`, `object ownership checks`, and `special-case exceptions`.

- Roles and tiers are projected in the access-control layer in [`access-control.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/access-control.ts).
- Route prehandlers enforce auth and sometimes named permissions, such as `requirePermission('knowledge.write')` or model-tier gates.
- Many handlers also manually query the DB to enforce row ownership for files, sessions, messages, workflows, and notebooks.
- Some routes intentionally relax this, such as public workflow execution.
- RAG routes use `optionalAuth` in some cases, creating different authorization behavior depending on endpoint and payload.
- Frontend feature availability is also shaped by `/api/me/access`, but that is advisory compared to backend enforcement.

This means the real authorization rule is usually:
- user must be authenticated
- user must have required role/tier capability
- user must own the resource, unless the route has a custom exception path

Enforcement Locations

Enforcement is spread across several layers rather than centralized.

- **Token verification and auth state hydration**
  -
[`middleware/auth.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/middleware/auth.ts)
- **Capability and tier logic**

  -
[`access-control.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/access-control.ts)
- **Route-level prehandlers**
  - auth required
  - optional auth
  - permission checks
  - model access checks
- **In-handler ownership checks**
  - especially in chat, files, RAG, workflows, notebooks, feedback
- **Frontend gating**
  -
[`AuthContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/AuthContext.tsx)
  - access snapshot-driven UI hiding/disabling

The strongest enforcement is in backend middleware plus route checks. The weakest part is object-level consistency, because many routes implement it manually.

Sensitive Operations

The most sensitive routes and operations are:

- Auth endpoints:
  - register, login, refresh, logout, password reset, OAuth callback
- Security endpoints:
  - 2FA setup/disable/verify, session/security settings
- File and knowledge operations:
  - upload, delete, retrieval, transcript/media insight access
- RAG endpoints:
  - search and chat, especially where unauthenticated or partially authenticated access is allowed
- Workflow execution:
  - executing public/private workflows and any node that touches private file resources
- User export/settings:
  - anything returning profile/security/export data
- Integrations:
  - OAuth callbacks, token storage, provider access-token refresh/use
- Admin or support-like access paths:
  - anything influenced by hardcoded privileged identities

These operations either mint trust, expose data, or trigger cross-resource actions.

Weak Security Signals

A few patterns stand out as unsafe or brittle.

- Refresh tokens appear too weakly tied to server-side revocable session state.

- Frontend token storage uses localStorage-style persistence with reversible client-side obfuscation, which is not meaningful protection against XSS or malicious extensions.
- Privileged behavior is partly derived from hardcoded email identity in [`access-control.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/access-control.ts#L31), which is a serious policy smell.
- Authorization is not uniformly centralized. Many routes rely on custom ownership logic, which increases omission risk.
- Some RAG endpoints use `optionalAuth`, which is dangerous in a system where document retrieval can expose sensitive text.
- Workflow definitions can contain references to private resources inside node payloads, but those references do not benefit from DB-level integrity or uniform per-execution reauthorization.
- `userSecurity.activeSessions` and `revokedTokens` suggest two overlapping session-truth models.
- OAuth/integration tokens are stored in app tables, and token-at-rest handling looks operationally sensitive.
- Security headers exist in code but appear inconsistently applied at runtime.
- The repo includes env/log artifacts, which is an operational hygiene warning.

Immediate Investigation Areas

1. Refresh-token revocation and password-reset invalidation.
Confirm whether a stolen refresh token remains usable after logout, password reset, or 2FA changes.

2. RAG auth boundaries.
Verify what an unauthenticated caller can retrieve through `/api/rag/search` and `/api/rag/chat`, especially when `fileIds` are omitted.

3. Public workflow execution.
Check whether executing a public workflow can indirectly access the creator’s private documents or media assets.

4. Hardcoded privilege escalation paths.
Remove or audit any email-based support/admin promotion logic.

5. Frontend token persistence.
Assess XSS exposure and move toward `HttpOnly`, `Secure`, `SameSite` cookie-based auth if feasible.

6. Integration token storage.
Review encryption-at-rest, key management, redaction, and access controls for `userIntegrations` and any BYOK material.

7. Route-by-route object authorization coverage.
Focus on chat exports, file reads, notebook document attachments, bookmarks, and any endpoints using polymorphic IDs or JSON-based resource references.

Overall, the auth model is functional and fairly full-featured, but it is not cleanly session-centric and not consistently policy-centric. The biggest risk is not missing JWT auth; it is the unevenness between strong central auth and scattered object/resource authorization rules.

Frontend Architecture Analysis Prompt

Frontend Architecture Analysis Prompt
Analyze the frontend architecture in this codebase.

Explain:
1. UI structure and feature organization
2. State management strategy
3. API interaction patterns
4. Routing/navigation model
5. Component reuse vs duplication
6. Validation and error handling in the UI
7. UX complexity hotspots
8. Areas where frontend is carrying business logic that belongs in backend

Return:
- Frontend Architecture
- State Model
- API Integration Patterns
- UX Complexity Areas
- Business Logic Leakage
- Refactor Opportunities

Frontend Architecture

The frontend is a `single React SPA with a workspace shell`, not a set of isolated micro-frontends. The main shell lives in [`App.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/App.tsx), where one authenticated layout wraps most product areas: chat, knowledge, RAG search, workflows, notebooks, media AI, docs, settings, and bookmarks.

Feature organization is mixed but readable:
- top-level route/page shells in [`pages/`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/pages)
- larger feature UIs in [`components/chat/`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/chat), [`components/rag/`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/rag), [`components/workflow/`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/workflow), [`components/knowledge/`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/knowledge), and [`components/notebook/`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/notebook)
- service wrappers in [`services/`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/services)
- state split across contexts, Zustand stores, and React Query

A few duplication signals stand out:
- both
[`ChatInterface.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/chat/ChatInterface.tsx) and
[`ChatInterfaceNew.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/chat/ChatInterfaceNew.tsx)
- both
[`KnowledgeManager.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/knowledge/KnowledgeManager.tsx) and
[`KnowledgeManagerNew.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/knowledge/KnowledgeManagerNew.tsx)
- both
[`UserRoleContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/UserRoleContext.tsx) and
[`EnhancedUserRoleContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/EnhancedUserRoleContext.tsx)

That suggests an app in active transition rather than a fully converged frontend architecture.

State Model

State management is layered, but not especially unified.

- `AuthContext` owns auth bootstrap, token refresh, user profile state, and login/logout in [`AuthContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/AuthContext.tsx)
- `EnhancedUserRoleContext` owns workspace mode, access snapshot, role-derived UI behavior, and onboarding defaults in [`EnhancedUserRoleContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/EnhancedUserRoleContext.tsx)
- React Query handles server-state fetching/caching globally via the `QueryClient` in [`App.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/App.tsx)
- Zustand stores hold feature-local UI/app state, especially chat and RAG chat sessions/messages in [`chatStore.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/stores/chatStore.ts) and [`ragChatStore.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/stores/ragChatStore.ts)
- persistent browser storage is used heavily for auth tokens, user info, role/workspace mode, and some feature state

The practical strategy is:
- React Query for fetched server data
- Zustand for interactive feature state
- Context for global app/session concerns
- localStorage-backed persistence for continuity

That works, but it also means the same concept can exist in multiple places. Chat sessions are a good example: React Query caches them, Zustand persists them, and route state also reflects the active session.

API Integration Patterns

API access is only partly standardized.

- The nominal common client is [`api-client.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/utils/api-client.ts), which adds auth headers, retry logic, and toast-based error handling.
- Feature service wrappers like [`workflowApi.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/services/workflowApi.ts) use that client fairly cleanly.
- `AuthService` bypasses the shared client and uses raw `fetch`, plus its own response normalization and error handling in [`authService.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/services/authService.ts).
- Endpoint definitions are centralized in [`config/api.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/config/api.ts), which helps with discoverability.

Patterns that matter:
- response normalization happens client-side because backend shapes are not fully consistent
- retries happen at the client level for some requests
- some services suppress global toasts so feature UIs can manage errors locally
- streaming and unload-time persistence use direct `fetch`/`sendBeacon` instead of the shared client, as seen in [`useChat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/hooks/useChat.ts)

This is a workable API layer, but it is not a strict API SDK. Different modules interpret backend contracts differently.

UX Complexity Areas

The main UX complexity hotspots are where the frontend has to coordinate long-lived or stateful workflows.

- **Chat**
  - session creation, optimistic local messages, streaming placeholders, unload-time completion/abandonment tracking, and route/session synchronization all happen in [`useChat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/hooks/useChat.ts)
- **Knowledge / file processing**
  - upload, reprocess, stage badges, bulk actions, document filters, and async status visibility make this area operationally dense

- **Workflow builder**
  - node palette, canvas, settings, execution polling/status, and template logic create a lot of client-side orchestration pressure
- **Access/workspace mode**
  - one user can have auth state, backend role, subscription tier, workspace mode, onboarding state, and UI feature flags all influencing navigation
- **Global shell communication**
  - `window` events like `toggleSidebar`, `navigateToView`, and `openUpgrade` in [`App.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/App.tsx) are a sign that some cross-feature coordination is escaping the component tree

Validation and UI error handling are also split:
- form/file/message/query validation utilities live in [`validation.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/utils/validation.ts)
- network errors often become toast notifications through the shared API client
- some services show their own toasts
- some background polls intentionally suppress toasts

That gives flexibility, but it also makes the UX less predictable.

Business Logic Leakage

The frontend is carrying more business logic than it should in a few important places.

- It reconstructs access policy locally. [`EnhancedUserRoleContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/EnhancedUserRoleContext.tsx) includes fallback permission maps, workspace-mode mappings, module-permission maps, and local entitlement defaults when `/api/me/access` fails.
- `ProtectedRoute` enforces role/subscription gating in the UI in [`ProtectedRoute.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/auth/ProtectedRoute.tsx). That is fine for UX, but it can drift from backend policy.
- `validation.ts` defines detailed product rules like password complexity, supported file types, filename rules, query safety, and content length in [`validation.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/utils/validation.ts). Some of that belongs server-side as the true source of policy.
- `useChat.ts` decides session completion vs abandonment semantics and transmits those state changes on unload. That is a business workflow rule, not just a view concern.
- `api-client.ts` rewrites some 403 errors into upgrade prompts based on message text in [`api-client.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/utils/api-client.ts). That means billing/product semantics are inferred from backend error strings.
- `AuthService` normalizes mismatched auth payload formats (`accessToken` vs `token`) in [`authService.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/services/authService.ts), which is really compensating for backend contract inconsistency.

Refactor Opportunities

- Consolidate state ownership. Pick one pattern for feature server state plus one for local UI state, and reduce overlap between React Query, Zustand, and persistent storage.
- Standardize API access. Move auth onto the shared client or a formal typed SDK so response normalization and error handling are not duplicated.
- Reduce frontend policy fallbacks. The UI should consume a server-issued access snapshot, not synthesize permissions and workspace entitlements when that snapshot fails.
- Eliminate duplicate generations of major features. The `*New` components and overlapping contexts are a cleanup target.
- Replace window-event coordination with explicit app-level state or routed actions.
- Narrow frontend validation to UX feedback and keep canonical product rules on the backend.
- Make async UX patterns more uniform across uploads, workflows, and chat so users see one consistent model of queued/running/completed/failed work.

Overall, the frontend is a capable but transitional SPA: strong feature breadth, decent caching and shell structure, but too many parallel state systems and too much product-policy logic living client-side.

Code Quality & Maintainability Prompt

Code Quality & Maintainability Prompt
Act as a senior engineering reviewer.

Assess this codebase for maintainability.

Look for:
1. Code duplication
2. Overloaded modules/files
3. Dead code or legacy residue
4. Inconsistent naming or abstractions
5. Tight coupling
6. Poor separation of concerns
7. Unclear ownership of logic
8. Signs that future development will become slow or risky

Return:
- Maintainability Strengths
- Maintainability Weaknesses
- Complexity Hotspots
- Technical Debt Signals
- Recommended Refactor Priorities

Maintainability Strengths

- The product is organized around recognizable domains: auth, chat, files, RAG, workflows, notebooks, preferences, subscriptions. Even though the boundaries are imperfect, the intent is readable.
- The backend has a real relational core in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts), and many important ownership chains are modeled explicitly.
- There are emerging orchestration layers, especially in [`chat-orchestrator.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/application/chat/chat-orchestrator.ts) and [`rag-chat-orchestrator.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/application/rag/rag-chat-orchestrator.ts), which is healthier than putting everything directly in routes.
- Async execution is not entirely ad hoc. The worker and workflow runner show deliberate effort to persist state and recover from long-running work.
- The frontend has a coherent SPA shell with shared providers, route guards, and a common endpoint catalog in [`services/frontend/src/config/api.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/config/api.ts).

Maintainability Weaknesses

- Several core files are overloaded and act like mini-systems:

  -
[`services/backend/src/routes/chat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts)
  -
[`services/backend/src/routes/files.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts)
  -
[`services/backend/src/services/queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts)
  -
[`services/backend/src/database/schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts)
- Logic ownership is blurry. Important business rules live across middleware, routes, services, repositories, workers, and frontend fallbacks rather than one obvious source of truth.
- The codebase tells multiple architectural stories at once: modular monolith, future microservices, worker sidecar, and enterprise multi-tenant platform. That increases drift and hesitation during changes.
- State is often duplicated:
  - DB status plus JSON metadata flags
  - backend permissions plus frontend-derived permission maps
  - React Query + Zustand + persisted local storage copies of related UI state
- Object authorization is not uniformly centralized, which makes security and maintainability problems reinforce each other.
- Heavy JSONB use in the backend means more behavior depends on conventions than on typed schema constraints.

Complexity Hotspots

- **Async pipeline layer**
  -
[`queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts) handles document processing, indexing, media jobs, workflow interactions, storage, and status mutation. It is a clear hotspot for regression risk.
- **Workflow execution**
  -
[`workflow-runner.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-runner.ts) and
[`workflow-executor.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-executor.ts) aggregate many subsystems and will become harder to reason about as node types grow.
- **Chat paths**
  - Standard chat, streaming chat, session management, message actions, and save-to-knowledge behavior are spread between routes, hooks, stores, and orchestrators.
- **Access and entitlement logic**
  - Permissions, roles, tiers, workspace mode, frontend gating, and ad hoc route checks are not consolidated enough.
- **Frontend feature transitions**

  - `ChatInterface` vs `ChatInterfaceNew`, `KnowledgeManager` vs `KnowledgeManagerNew`, and duplicate role contexts indicate in-progress migration that future contributors will have to mentally untangle.

Technical Debt Signals

- Duplicate or legacy residue in the frontend:
  - [`ChatInterface.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/chat/ChatInterface.tsx)
  - [`ChatInterfaceNew.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/chat/ChatInterfaceNew.tsx)
  - [`KnowledgeManager.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/knowledge/KnowledgeManager.tsx)
  - [`KnowledgeManagerNew.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/components/knowledge/KnowledgeManagerNew.tsx)
  - [`UserRoleContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/UserRoleContext.tsx)
  - [`EnhancedUserRoleContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/EnhancedUserRoleContext.tsx)
- Backend schema/application drift:
  - notebook cell enum mismatch between [`notebooks.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/notebooks.ts#L21) and [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts#L568)
  - raw `audit_logs` usage outside the typed schema in [`audit.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/utils/audit.ts)
- Inconsistent abstractions:
  - some APIs use service wrappers, some use raw fetch
  - some business logic sits in application orchestrators, some directly in routes
  - some async state is modeled explicitly, some through metadata conventions
- Hidden coupling through shared tables:
  - `files` and `documentChunks` are touched by chat, RAG, workflows, media, notebooks
  - `users` and access side tables affect almost every feature
- Pseudo-microservice residue like [`docker-compose.microservices.yml`](/Users/aeishwary/JoaLLM-platform/joallm-platform/docker-compose.microservices.yml) can confuse the actual runtime model.

These are all signs that future development will get slower because every change requires more cross-module context and more “is this the old path or the new one?” judgment.

Recommended Refactor Priorities

1. Break up overloaded core files first.
Start with `queue.ts`, `chat.ts`, and `files.ts` by extracting stage-specific services and smaller route handlers. That will lower change risk immediately.

2. Centralize policy logic.
Consolidate access/permission/resource-ownership checks and remove frontend fallback policy synthesis where possible.

3. Eliminate duplicate frontend generations.
Pick the surviving chat, knowledge, and role-context implementations, migrate callers, and remove the shadow versions.

4. Make async state models explicit.
Replace metadata-driven workflow/file stage conventions with clearer typed transition helpers and smaller stage modules.

5. Reduce contract normalization in the frontend.
Standardize backend response shapes so the UI does less defensive adaptation.

6. Tighten schema/application alignment.
Bring raw-managed tables into the typed schema, align enums and validators, and reduce behaviorally important JSON blobs where possible.

7. Clarify domain ownership.
For each major feature, decide where truth lives:
- route vs orchestrator
- DB status vs metadata
- frontend entitlement vs backend entitlement
- chat message vs generic inference log

Overall, the codebase is still workable, but it’s at the point where feature growth without structural cleanup will start compounding cost. The biggest maintainability risk is not “bad code” in isolation; it’s too many partially overlapping paths and too many important rules without one obvious home.

Testability Analysis Prompt

Testability Analysis Prompt
Analyze this codebase for testability.

Determine:
1. Which parts are easy to test
2. Which parts are hard to test and why
3. Hidden dependencies that make testing difficult
4. Missing seams for mocking or isolation
5. Whether architecture encourages or resists automated testing
6. The minimum viable test strategy for this codebase

Return:
- Testable Areas
- Hard-to-Test Areas
- Causes of Poor Testability
- Suggested Test Pyramid
- Refactor Suggestions for Better Testability

Testable Areas

These parts are the easiest to test because they have narrower dependencies or mostly deterministic logic.

- Pure frontend utilities and formatters:
  - [`services/frontend/src/utils/validation.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/utils/validation.ts)
  - file/query/message validation helpers
- Frontend stores:
  - [`services/frontend/src/stores/chatStore.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/stores/chatStore.ts)
  - [`services/frontend/src/stores/ragChatStore.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/stores/ragChatStore.ts)
  - Zustand actions and persistence behavior are straightforward unit-test targets
- Access/role mapping logic on the frontend:
  - workspace-mode normalization and permission mapping in [`EnhancedUserRoleContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/EnhancedUserRoleContext.tsx)
- Small backend helpers and repositories with focused logic:
  - response/DTO normalization
  - lineage helper behavior
  - permission projection logic in [`services/backend/src/services/access-control.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/access-control.ts), although it still mixes policy with special cases

- Schema-level behavior that can be checked with DB-backed integration tests:
  - FK cascades
  - uniqueness rules
  - one-row-per-user side tables
  - workflow execution status persistence

These areas fit unit or narrow integration tests without needing the whole stack alive.

Hard-to-Test Areas

These are hard because they couple persistence, async work, external providers, and business policy in the same path.

- **Queue and background processing**
  - [`services/backend/src/services/queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/queue.ts)
  - It touches DB, Redis/BullMQ, storage, document parsing, embeddings, media processing, workflow resume, and multiple tables at once.
- **Workflow execution**
  - [`services/backend/src/services/workflow-runner.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-runner.ts)
  - [`services/backend/src/services/workflow-executor.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/services/workflow-executor.ts)
  - High branching, graph execution, checkpointing, async suspension, and node-type-specific side effects make this difficult to isolate.
- **Chat end-to-end**
  - standard chat plus streaming chat are split across routes, orchestrators, provider calls, stores, and session persistence
  - streaming especially is awkward because client-visible output and DB writes happen in different phases
- **File upload to indexed knowledge**
  - upload, storage, queue handoff, chunking, indexing, and status mutation are spread across route handlers and workers
- **RAG flows**
  - retrieval quality depends on embeddings, query enhancement, DB state, permissions, optional auth, and sometimes multiple retrieval passes
- **Auth flows**
  - login/refresh/logout/2FA/OAuth involve JWTs, Redis, DB state, Google, and browser storage assumptions
- **Frontend route/auth shell**
  - auth bootstrap and access snapshot fallback logic depend on browser storage, fetch behavior, and async initialization timing

These are not impossible to test, but they resist cheap, fast, isolated tests.

Causes of Poor Testability

The main issue is not lack of logic separation everywhere. It is that important flows cross too many layers at once.

- Shared global dependencies:
  - most backend code imports the shared `db`
  - queue code imports concrete providers directly
  - auth code reaches into storage, JWT, DB, Redis, and middleware state
- Large orchestration files:
  - routes often mix validation, permission checks, DB operations, provider calls, and response shaping
- Weak mocking seams:
  - concrete services are often imported directly rather than injected
  - worker paths and provider integrations are not consistently behind small interfaces at the call site
- Async state is partly implicit:
  - status in one field, stage flags in JSON metadata, queue state elsewhere
  - this makes assertions more brittle
- Duplicate paths:
  - sync chat vs streaming chat
  - legacy/new frontend components
  - auth service using raw fetch while other services use the shared API client
- External dependency entanglement:
  - OpenAI/Anthropic/Groq/Cohere/Ollama
  - Google OAuth
  - Redis/BullMQ
  - FFmpeg/media tooling
  - object/local storage backends

Hidden dependencies that complicate tests:
- browser storage and dev auto-login env behavior in [`AuthContext.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/contexts/AuthContext.tsx)
- `window` events and `sendBeacon` usage in [`useChat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/hooks/useChat.ts)
- direct DB mutations from routes and workers instead of one domain façade
- email-based special-case role logic in access control, which creates policy behavior outside obvious test inputs

Overall, the architecture partially encourages testing at the utility/repository level, but resists it at the most important workflow level.

Suggested Test Pyramid

Minimum viable test strategy for this codebase should be pragmatic, not exhaustive.

1. **Unit tests**
- frontend validation utils
- Zustand stores
- access/permission projection
- small pure helpers for workflow graph prep, payload normalization, and state mapping
- API client error normalization

2. **Backend integration tests against a real test Postgres**
- auth flows: register/login/refresh/2FA happy path and major denial cases
- chat session/message persistence
- file upload metadata creation
- workflow execution row lifecycle
- permissions and ownership checks on sensitive routes
- schema integrity: cascade delete, uniqueness, nullable edge cases

3. **Service-level integration tests with mocked providers**
- chat orchestrator with mocked LLM provider
- RAG orchestrator with mocked retrieval/provider layer
- workflow runner with mocked node executors and mocked async resume trigger
- file processing service with mocked parser/storage/embedding components

4. **A few end-to-end tests**
- login -> chat send -> session appears
- upload file -> processing status progresses
- execute workflow -> execution record reaches terminal state
- access gating for a lower-tier user

5. **Very small async/worker test suite**
- one happy-path document processing job
- one media-processing chain
- one suspended workflow resume case

That is enough to cover the highest-risk behavior without trying to fully simulate every provider and pipeline branch.

Refactor Suggestions for Better Testability

- Introduce dependency injection at orchestration boundaries.
  - Chat, RAG, workflow, and queue handlers should accept provider/storage/retrieval interfaces instead of importing concrete implementations directly.
- Split overloaded files into stage-oriented modules.
  - `queue.ts`, `chat.ts`, and `files.ts` are the best first targets.
- Create explicit domain services for state transitions.
  - Example: `FileLifecycleService`, `WorkflowExecutionService`, `ChatSessionService`.
  - This gives tests one place to assert business transitions.
- Unify duplicated execution paths where possible.
  - Streaming and non-streaming chat should share more core logic.
- Replace metadata-driven assertions with typed transition helpers.

  - Especially for file processing and workflow execution.
- Add a test harness for infrastructure-heavy components.
  - standard factories for test DB, fake queue, fake storage, fake provider responses
- Move frontend business fallback logic behind one access adapter.
  - That reduces the number of places that must be mocked in UI tests.

Net assessment: the codebase is testable around the edges and hard to test at its operational core. A focused integration-heavy strategy, plus a few structural seams around orchestrators and queues, would give the biggest payoff fastest.

Refactor Readiness Prompt

Refactor Readiness Prompt
Act as an architect planning the next stage of system evolution.

Based on the current codebase, assess refactor readiness.

Evaluate:
1. Which areas are stable enough to leave alone
2. Which areas need cleanup before new features
3. Which areas should be modularized
4. What should be extracted first if moving toward services
5. What refactors are high-value, low-risk
6. What refactors are tempting but dangerous

Return:
- Stable Zones
- Fragile Zones
- Refactor Candidates
- Extraction Candidates
- Safe Refactor Sequence
- Dangerous Refactors to Avoid

Stable Zones

These parts look stable enough to keep mostly intact for now.

- The overall `modular monolith + worker sidecar` shape is still a good fit for the current codebase. Forcing service decomposition now would add coordination cost before enough boundaries exist.
- The core relational model around `users`, `chatSessions`, `messages`, `files`, `documentChunks`, `workflows`, and `workflowExecutions` is worth preserving, even if some tables need tightening in [`schema.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/database/schema.ts).
- The frontend app shell and route-based product structure in [`App.tsx`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/frontend/src/App.tsx) are serviceable.
- The worker-based async approach is directionally right. The problem is not “queues exist,” it is how much unrelated logic is concentrated in them.
- The use of orchestrators for chat and RAG is a good pattern to build on rather than replace.

Fragile Zones

These are the places where adding features before cleanup will compound risk quickly.

- [`services/backend/src/services/queue.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platfor

m/services/backend/src/services/queue.ts) is too overloaded. It is handling multiple pipelines, multiple data domains, and workflow resumption.
-
[`services/backend/src/routes/chat.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/chat.ts) and
[`services/backend/src/routes/files.ts`](/Users/aeishwary/JoaLLM-platform/joallm-platform/services/backend/src/routes/files.ts) carry too much direct business logic.
- Access control is functionally important but structurally scattered across middleware, route checks, access service, and frontend fallbacks.
- File and workflow state modeling is fragile because status is split across top-level fields and metadata JSON.
- Frontend feature duplication and partial migrations (`ChatInterface` vs `ChatInterfaceNew`, `KnowledgeManager` vs `KnowledgeManagerNew`, role context overlap) will slow every UI change.
- Any area where the frontend synthesizes policy when backend access bootstrap fails is fragile by definition.

Refactor Candidates

High-value modularization targets inside the current monolith:

- **File lifecycle module**
  - Separate upload intake, text extraction, chunk generation, indexing, and media enrichment into distinct services with explicit stage contracts.
- **Workflow execution module**
  - Split graph scheduling, node execution, suspension/resume, and persistence/logging into smaller internal modules.
- **Access policy module**
  - Centralize role/tier/object access checks and stop letting routes invent their own patterns.
- **Async job state module**
  - Introduce a uniform job/state abstraction for uploads, indexing, media, and workflow execution.
- **Frontend API layer**
  - Unify `fetch` usage behind one typed API client/SDK and reduce per-service contract normalization.
- **Frontend access layer**
  - Make backend `/api/me/access` the single source of truth and remove local fallback permission synthesis where possible.

These are modularization refactors, not service extractions. They reduce risk without forcing distributed systems complexity yet.

Extraction Candidates

If the team does move toward services, the best first candidates are the ones that already behave like bounded async capabilities.

1. **Media processing**

- It already acts like a specialized compute pipeline.
- It has clearer inputs/outputs than most business domains.
- It is expensive and operationally distinct.

2. **Document processing/indexing worker path**
- Not the full files domain, but the extraction/chunking/embedding pipeline is a natural service boundary.
- The API can continue owning file records while a processor service owns the heavy lifting.

3. **Telemetry / lineage / audit ingestion**
- High fan-in, lower transactional coupling.
- Good candidate for async/event-driven extraction.

4. **Model catalog / model metadata**
- Small, read-heavy, low-risk if separated.

What should not be extracted first:
- auth/access
- workflows
- full chat domain
- full knowledge/RAG domain

Those are too entangled with shared state and cross-feature behavior today.

Safe Refactor Sequence

1. **Stabilize contracts**
- Standardize backend response shapes.
- Align schema and route validators.
- Remove obvious frontend duplicate implementations.

2. **Centralize policy**
- Consolidate access checks and entitlement logic.
- Replace frontend permission fallbacks with backend-issued truth.

3. **Decompose overloaded internals**
- Break `queue.ts`, `chat.ts`, and `files.ts` into smaller stage/domain modules.
- Add explicit state transition helpers for files and workflow executions.

4. **Create infrastructure seams**
- Introduce cleaner interfaces for storage, providers, retrieval, and queue submission.
- This improves testability and prepares service extraction later.

5. **Normalize async state**
- Add a more uniform async job/status model across uploads, workflows, and media processing.

6. **Extract specialized async compute first**

- Media processing, then document processing/indexing, then telemetry if needed.

That order gives the team immediate maintainability wins and makes later extraction much less painful.

Dangerous Refactors to Avoid

- **Splitting workflows into a service right now**
  - It consumes too many other subsystems and would turn in-process coupling into distributed coupling.
- **Extracting auth/access too early**
  - The code still depends on identity and entitlement data everywhere; premature extraction would create constant cross-service policy calls.
- **Breaking out chat before unifying chat paths**
  - Streaming and non-streaming logic still diverge too much.
- **A “microservices rewrite” based on folder intent**
  - The live code is still a modular monolith. Reorganizing by desired architecture without first cleaning actual coupling would mostly move complexity, not reduce it.
- **Front-to-back concurrent rewrites**
  - There is already frontend duplication and backend drift. Refactoring both architecture layers at once would make the system harder to stabilize.
- **Schema-first normalization everywhere at once**
  - Some JSON-heavy areas do need normalization, but broad schema surgery without narrowing ownership first could create migration churn and break active features.

The short version is: leave the monolith shape alone, clean the policy and async internals, remove duplicate frontend paths, then extract specialized background capabilities only after the internal seams are real.

Explain It Like I Own the Product” Prompt

Explain It Like I Own the Product” Prompt
Explain this codebase as if I am the product owner and system architect responsible for its future.

I do not want file-by-file explanation.

Tell me:
1. What kind of product this really is
2. What capabilities it already has
3. What technical bets it has made
4. What operational risks are embedded in it
5. What kind of team/process this codebase needs
6. What will break first under scale, complexity, or enterprise usage

Be direct, opinionated, and evidence-based.

What Product This Really Is

This is not “just an AI chat app.” It is an `AI work platform` trying to unify:

- multi-model chat
- knowledge ingestion and retrieval
- workflow automation
- notebook-style AI workspaces
- media analysis
- enterprise-ish access, billing, and integrations

In product terms, it is aiming to be a shared operating surface for AI-assisted work, not a single killer feature. The real center of gravity is: `bring in data, run AI against it, operationalize that through workflows, and track usage/access around it`.

That’s ambitious, and the code reflects it. The platform already behaves more like an internal AI productivity suite than a narrow SaaS feature.

What It Already Has

It already has more capability than many teams would admit from the outside:

- account system with email/password, Google auth, refresh tokens, and 2FA
- tiered access model with roles, plans, permissions, and usage limits
- full chat system with sessions, streaming, exports, message actions, and saved context
- document and media upload pipeline
- chunking, embeddings, vector-backed retrieval, and grounded RAG chat/search
- workflow builder plus persisted execution state and async resume
- notebook layer for structured AI work
- bookmarks, feedback capture, training-consent scaffolding, usage telemetry, lineage, and analytics

- storage abstraction, queue workers, and multiple model providers
- payment/integration hooks and enterprise-flavored workspace concepts

So the product is already broad. The issue is not missing surface area. The issue is that the platform is wider than it is deep in some critical areas.

What Technical Bets It Has Made

The codebase has made a few big bets, whether intentionally or not.

1. **Modular monolith over true services**
It has chosen one main backend with shared DB ownership, plus a worker sidecar. That is the right bet for this stage. The repo may talk microservices in places, but the real architecture is still one backend.

2. **Postgres as the center of truth**
A lot of system behavior is anchored in Postgres: users, chat, files, chunks, workflows, notebook state, preferences, feedback, lineage. That is a strong, practical choice, but it means DB health and schema discipline are existential.

3. **Redis/BullMQ for operational elasticity**
Async work is pushed into queues for ingestion, indexing, media, and workflow execution. Also the right direction. The problem is that the queue layer is becoming a second application runtime, not just a background helper.

4. **Multi-provider AI abstraction**
The platform is betting that model/provider flexibility matters. That's smart commercially and operationally, but it increases branching and test complexity.

5. **JSON-heavy extensibility**
The system uses JSONB for workflow graphs, metadata, execution logs, UI configs, session state, and more. That speeds iteration early, but it weakens invariants and makes long-term governance harder.

6. **Frontend as an active policy participant**
The frontend is not just rendering. It carries access assumptions, fallback permission logic, session semantics, and API normalization. That's a convenience bet now, and a consistency risk later.

What Operational Risks Are Embedded In It

There are several, and they are not minor.

- The system is highly dependent on `one backend + one database + one queue/cache layer`.
  If Postgres or the backend misbehaves, most product capabilities fail together.
- Authorization is not consistently centralized.

That is a serious enterprise risk because access logic is scattered across middleware, route handlers, and frontend gating.
- Async state is fragmented.
  Files, workflows, and media all track progress differently. That creates stuck states, support pain, and hidden data drift.
- Security posture is uneven.
  The code has real auth features, but also dangerous shortcuts and soft spots: local token persistence, hardcoded privileged identity behavior, uneven resource authorization, and integration token handling concerns.
- The data model is strong at the core and weak at the edges.
  Polymorphic references, JSON-backed relationships, and duplicate status fields will cause integrity issues over time.
- The API surface is broad but not contract-disciplined.
  It lacks strong versioning, consistent response shapes, and uniform async semantics.
- The frontend is in transition.
  Duplicate implementations and overlapping contexts mean UI changes will get slower and riskier unless cleaned up.

This is the kind of platform that can look stable in demos while accumulating operational fragility underneath.

What Kind of Team/Process This Codebase Needs

This codebase needs a `platform-minded product engineering team`, not just fast feature shipping.

Specifically:

- at least one strong backend/platform engineer who owns data, async workflows, and reliability
- one frontend engineer who can simplify the state model and remove duplicate UI paths
- one security-minded engineer or regular security review cadence
- someone acting as technical product integrator across chat, knowledge, workflows, and enterprise features

Process-wise, it needs:

- architecture discipline on where logic lives
- explicit API and state-transition contracts
- integration-heavy test coverage, not just unit tests
- regular schema and permission audits
- release discipline for async and data-touching changes
- feature work sequenced with structural cleanup, not layered endlessly on top

This is no longer a “two builders hacking fast” codebase unless the ambition is reduced. It has crossed into territory where unowned complexity will slow delivery and raise incident risk.

What Will Break First

Under `scale`:
- the queue/worker pipeline and file/media processing path
- large route/service files that already act as orchestration hubs
- anything that relies on in-memory assumptions around streaming, polling, or session state
- Postgres load from mixed transactional + retrieval + telemetry usage

Under `complexity`:
- authorization consistency
- async status correctness
- workflow execution behavior as more node types are added
- frontend state coherence across React Query, Zustand, local storage, and route state
- developer velocity, because too many features share the same core entities

Under `enterprise usage`:
- access control and tenant isolation assumptions
- auditability and data-governance expectations
- token/session management
- export/privacy boundaries
- “public workflow with private resources” style edge cases
- supportability when async jobs stall or partial states accumulate

The first thing to break is not likely raw inference. It’s more likely `control-plane trust`: who can access what, whether background work finishes correctly, and whether the system’s stored state still matches reality.

Bottom Line

This is a serious product foundation with real breadth and real ambition. The architecture choice is still mostly right for now. The danger is that the platform already behaves like a company-sized system while still carrying startup-era shortcuts and transitional structure.

If you want this to become enterprise-capable and sustainably extensible, the next phase should not be “add more major features.” It should be:

- tighten access and security
- simplify ownership of state and policy
- break up async orchestration hotspots
- remove duplicate frontend/backend paths
- standardize contracts and lifecycle models

If you do that, the platform can grow. If you don’t, complexity will start compounding faster than feature value.

Master Prompt

Act as a principal architect performing a full codebase intelligence review.

Analyze this codebase deeply and do not merely summarize files.

Your objectives:
1. Explain what the system does at business and technical levels
2. Identify the real architecture, not the intended one
3. Map the main modules, execution flows, and data flows
4. Extract the core business logic and state transitions
5. Identify dependencies, external integrations, and infrastructure assumptions
6. Highlight maintainability, coupling, and technical debt issues
7. Flag fragile areas, unclear abstractions, and hidden risks
8. Assess security surface, async behavior, and testability
9. Recommend the most sensible next steps for stabilization and evolution

Return in these sections:
- Executive Summary
- Business Purpose
- Actual Architecture
- Core Modules
- Critical Flows
- Data Model and State Management
- Dependency / Infra Map
- Security and Access Model
- Async / Background Processing
- Maintainability Review
- Technical Debt and Fragility
- Refactor Readiness
- Top 10 Immediate Concerns
- Top 10 Strategic Recommendations

Be brutally honest. Infer from actual code behavior, not naming conventions.